

MARFT: Multi-Agent Reinforcement Fine-Tuning

Junwei Liao^{1,2}, Muning Wen¹, Jun Wang³, Weinan Zhang^{1,2}

¹Shanghai Jiao Tong University, ²Shanghai Innovation Institute, ³OPPO Research Institute
 {jwliao.ai, muningwen, wnzhang}@sjtu.edu.cn, {junwang.lu}@gmail.com

Abstract

Large Language Model (LLM)-based Multi-Agent Systems (LaMAS) have demonstrated remarkable capabilities in addressing complex, agentic tasks requiring multifaceted reasoning and collaboration, from generating high-quality presentation slides to conducting sophisticated scientific research. Meanwhile, Reinforcement Learning (RL) has been widely recognized for its effectiveness in enhancing agent intelligence, but limited research has investigated the fine-tuning of LaMAS using foundational RL techniques. Moreover, the direct application of conventional Multi-Agent Reinforcement Learning (MARL) methodologies to LaMAS introduces significant challenges, stemming from the unique characteristics and mechanisms inherent to LaMAS. To address these challenges, this article presents a comprehensive study of LLM-based MARL and proposes a novel paradigm termed **Multi-Agent Reinforcement Fine-Tuning (MARFT)**. We introduce a brand-new MG called Flex-MG, which aligns with the LaMAS optimization in real-world applications and a universal algorithmic framework tailored specifically for LaMAS, outlining the conceptual foundations, key distinctions, and practical implementation strategies. We begin by reviewing the evolution from traditional RL to Reinforcement Fine-Tuning (RFT), setting the stage for a parallel analysis in the multi-agent domain. In the context of LaMAS, we elucidate critical differences between classical MARL and MARFT, such as asynchronous agent interactions, profile-aware agent design, and heterogeneous architectural configurations etc. These differences motivate a transition toward a novel, LaMAS-oriented formulation of RFT. Central to this work is the presentation of a robust and scalable MARFT framework. We detail the core algorithm, emphasizing its modularity and adaptability, and provide a complete, open-source implementation to facilitate adoption and further research. The latter sections of the paper explore real-world application perspectives and opening challenges in MARFT, including dynamic environment modeling, sample inefficiency, and the current lack of cohesive frameworks. By bridging theoretical underpinnings with practical methodologies, this work aims to serve as a roadmap for researchers seeking to advance MARFT toward resilient, adaptive, and human-aligned solutions in agentic systems. Our implementation of the proposed framework is publicly available at: <https://github.com/jwliao-ai/MARFT>.

1 Introduction

Large Language Models (LLMs) are increasingly being deployed as a new generation of autonomous agents capable of performing agentic tasks—those that require decision-making, reasoning, and interaction with complex and dynamic environments [36, 30, 68]. These LLM-based agents are rapidly reshaping human-machine interaction and expanding the frontiers of autonomous systems. In addition to their strong natural language understanding and generation capabilities [12], LLMs can perform retrieval-augmented generation (RAG) [44], and, when integrated with external tools or APIs, can accomplish more sophisticated tasks on computers and mobile platforms [18, 119].

Furthermore, LLMs have been successfully embedded into embodied and simulated environments, functioning as agents such as robots or game players [90, 51, 8]. Their ability to comprehend instructions, learn from feedback, and generate context-aware outputs enables effective deployment across various domains, including healthcare, education, and software development [13, 11].

Reinforcement Learning (RL) is a machine learning paradigm in which an agent learns optimal behaviors through trial-and-error interactions with an environment to maximize cumulative reward. Unlike supervised or unsupervised learning, RL is driven by feedback in the form of rewards or penalties [85]. The recent release of OpenAI’s o1 model, trained using large-scale RL techniques, has reignited interest in the field due to its impressive reasoning abilities [61]. Following this, OpenAI expanded its broader reinforcement fine-tuning (RFT) research program focused on refining LLMs through RL approaches [62]. Distinct from conventional RL, RFT involves fine-tuning pretrained models using a relatively small set (dozens to thousands) of high-quality interaction trajectories. This process must maintain the language capabilities of the LLM while enhancing its reasoning and decision-making skills. Recent studies have shown that RFT can significantly improve the performance of LLM-based agents [15, 69, 76, 116, 97]. Notably, GRPO was proposed as a variant of Proximal Policy Optimization (PPO), enhancing reasoning abilities and revealing intriguing phenomena such as the “Aha moment” during large-scale RL training [76]. Building on this, DAPO and VAPO extend GRPO with additional techniques [114, 115]. Collectively, these efforts highlight the centrality of the trial-and-error nature of RL in advancing LLM intelligence. Despite these advances, directly applying RFT to multi-agent LLM settings remains underexplored. Naively transferring single-agent RFT techniques to multi-agent contexts often leads to challenges such as unstable training, inactive agents, and inefficient communication. These issues underscore the need to revisit and adapt established concepts from MARL to bridge the gap between RFT and LLM-based Multi-Agent Systems (LaMAS).

Multi-Agent Systems (MAS) have consistently demonstrated superior capabilities in addressing complex agentic tasks relative to their single-agent counterparts. This is evident in the latest General AI Assistants (GAIA) leaderboard [54], where the top-performing systems are all multi-agent frameworks. Prominent frameworks, including OpenAI Agents SDK (formerly Swarm), Microsoft AutoGen [105], Magnetic-One, CAMEL-AI OWL [6], and Google’s AI Co-Scientist [24], further reinforce the growing importance of multi-agent architectures. Notably, in OpenAI’s vision for AGI, multi-agent systems constitute the highest organizational layer. However, multi-agent systems are prone to a variety of failure modes—fourteen in total—related to system design, coordination, and control, where MARL is poised to address these challenges [9]. Besides, a recent framework systematically quantifies the contributions of different modules in multi-agent systems, potentially illuminating the direction for future multi-agent optimization in LLMs [111].

Multi-Agent Reinforcement Learning (MARL) extends standard RL to settings where multiple autonomous agents interact and learn simultaneously to achieve individual or shared objectives [100, 5, 80, 93]. Applications of MARL span numerous domains, including swarm robotics [34, 53], autonomous driving [75], strategic games [79], traffic management [39], and distributed resource allocation [59]. MARL methods can be broadly categorized into independent learning, joint learning, and coordination-based strategies [88, 52, 70]. A widely adopted paradigm is centralized training with decentralized execution (CTDE), with prominent algorithms including MADDPG [47], MAPPO [113], and HAPPO [43]. More recently, Multi-Agent Transformer (MAT) reframes MARL as a series of sequential decision-making problems, potentially opening new directions for research [101]. Despite these advances, integrating LLMs into MARL settings introduces significant challenges. Unlike conventional multi-agent systems in traditional MARL tasks, LaMAS are highly flexible and complex, particularly in their communication dynamics. Addressing the challenges requires rethinking fundamental assumptions in MARL, such as agent homogeneity and communication protocols, to accommodate the unique properties of LaMAS. Though there are easy-to-use and well-established frameworks such as MALib [122] unifying many existing MARL methods for population-based RL and OpenRLHF [32] providing implementations for most single-agent RL fine-tuning techniques, the absence of a unified framework that integrates LLMs as agents in dynamic environments with agen-

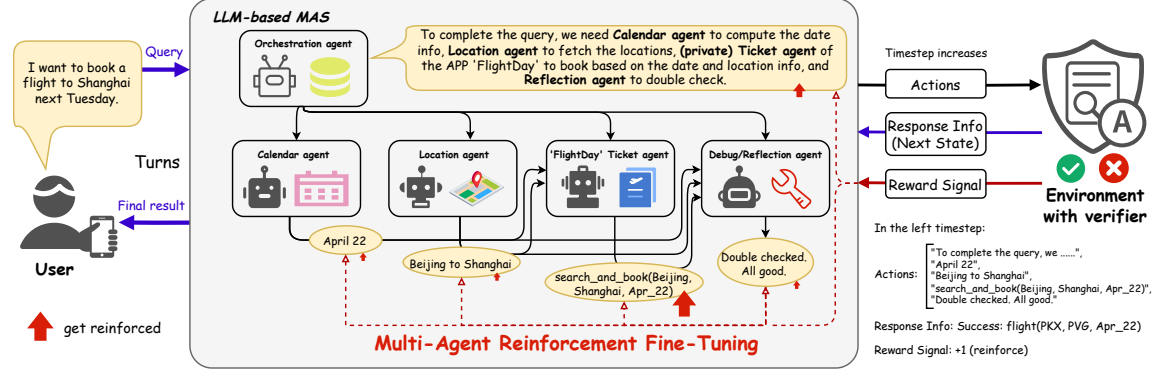


Figure 1: Illustration of MARFT in real-world agentic problem-solving scenarios.

tic tasks still constrains the swarm intelligence of LaMAS. Consequently, there remains a need for a well-established LLM-based multi-agent framework both theoretically and practically to unleash LaMAS’s swarm intelligence and potential better.

Given the accelerating progress in both RFT for LLMs and LaMAS, we are motivated to introduce and formalize the concept of **Multi-Agent Reinforcement Fine-Tuning (MARFT)**. This article aims to provide a comprehensive foundation for MARFT, serving both academic and industrial audiences. The remainder of this article is organized as follows: Section 2 revisits foundational concepts in RFT, LaMAS, and MARL. Section 3 gives the problem statement and formulation and outlines the key distinctions between MARFT and existing approaches. Section 4 introduces the MARFT methodology and its basic implementation strategies. Section 5 presents updated experiments on DeepScaler and DeepCoder, including comparisons with Independent PPO (MAPoRL) and ablations on role specification. Finally, Sections 6 and 7 provide future perspectives and highlight unresolved challenges, followed by an overall conclusion of the paper in Section 8.

2 Literature Review

This section presents a comprehensive review of topics closely related to MARFT, namely RFT, LaMAS, and MARL. We begin with a discussion of the algorithmic foundations and methodological formulations of RL, followed by an in-depth review of RFT. Subsequently, we provide an overview of LaMAS and MARL from a broader perspective, thereby laying the conceptual groundwork for the introduction and development of MARFT.

2.1 Reinforcement Fine-Tuning (RFT)

RFT is a fine-tuning paradigm that harnesses the capabilities of Reinforcement Learning to refine agent behavior. While it shares foundational similarities with traditional RL, RFT places greater emphasis on achieving expert-level performance in specific tasks using limited, high-quality data and a constrained number of optimization iterations. To provide context for the discussion of RFT, we first briefly revisit the core principles, mathematical formulations, and algorithmic methodologies of standard RL.

2.1.1 Language-augmented Markov Decision Process

In most sequential decision-making scenarios, problems are typically modeled as a *Markov Decision Process (MDP)* $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma \rangle$. In linguistic contexts, this framework can be further extended to a more specific form of language-augmented *Partial Observable Markov Decision Process (POMDP)* represented as $\langle \mathcal{V}, \mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma \rangle$ [94, 8]. Here, $\mathcal{V}$ represents the vocabulary of a language model, which facilitates natural language understanding within an LLM. The state space $\mathcal{S}$ can be situated in a semantic space, particularly in linguistic contexts. The observation function $\mathcal{O}: \mathcal{S} \rightarrow \mathcal{V}^L$ and action space $\mathcal{A} \subseteq \mathcal{V}^L$ respectively denote the observations and actions, both of which are explicitly

represented by sequences of L tokens from the vocabulary $w \in \mathcal{V}$. Furthermore, the action $a \in \mathcal{A}$ can be grounded to a specific tool or API call in real-world environments [71, 106, 48]. The transition function $\mathcal{T}: \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}'$, often represented as a probability distribution $\mathcal{P}$ in the form of $P(s'|s, a)$ describes the state transition following a specific action taken in a given state. However, in linguistic settings, the transition for observations can be directly achieved through sentence concatenation. The reward function $\mathcal{R}: \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$, typically denoted as $R(s, a, s')$, quantifies the reward for taking a particular action that induces a state transition. The discount factor γ reflects the diminishing value of rewards received over time. This constitutes the fundamental language-augmented POMDP, where a single LLM takes actions within a language-enhanced environment. For any other language-based tasks, specific POMDPs can be derived from this basic language-augmented POMDP.

2.1.2 Reinforcement Learning (RL)

The Objective of RL In classic reinforcement learning settings, an agent is capable of perceiving its environment, making decisions, and taking actions, while learning from past experiences to enhance its performance. The agent interacts with the environment until certain terminal conditions are met, generating a trajectory $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$ of length T , with a cumulative reward expressed as $g = \sum_{t=0}^T \gamma^t r_t$. The primary objective is for the agent to achieve a high cumulative reward in a dynamic and uncertain environment. This is precisely what RL aims to accomplish—obtaining a policy that enables the agent to dynamically act in dynamic environments to maximize cumulative reward. Mathematically, the objective is formulated as $\max_{\pi} \mathbb{E}_{\tau \sim p_{\pi}(\tau)} \left[\sum_{t=0}^T \gamma^t R_t \right]$ [85]. Here, the cumulative reward from timestep t is denoted as $G_t = \sum_{t'=t}^{\infty} \gamma^{t'-t} R_{t'}$.

Value-based Methods Given that RL optimizes the policy by leveraging rewards to incentivize the policy to learn actions that yield the highest rewards, it is natural to consider a value-based approach. This value-based approach provides an elegant and effective solution to the critical credit assignment problem [55, 85]. This approach focuses on learning indirect rewards, i.e., values, which estimate the goodness of an action before receiving the reward and ideally can better optimize the policy in long-horizon and reward-sparse tasks. To this end, two vital proxies to the RL objective are introduced: the state-action value function (Q-function) and the state value function (V-function) [85]. Their definitions are presented respectively as

$$Q^{\pi}(s_t, a_t) := \mathbb{E}_{\tau \sim p_{\pi}(\tau|s_t, a_t)} \left[\sum_{t'=t}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'}) \right] = \mathbb{E}_{(s,a)_{t+1:\infty} \sim p_{\pi}(s,a)} [G_t | s_t, a_t],$$

$$V^{\pi}(s_t) := \mathbb{E}_{\tau \sim p_{\pi}(\tau|s_t)} \left[\sum_{t'=t}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'}) \right] = \mathbb{E}_{a_t \sim \pi(a_t|s_t), (s,a)_{t+1:\infty} \sim p_{\pi}(s,a)} [G_t | s_t].$$

Here p_{π} is the trajectory distribution sampled using the policy π . Obviously, from their naming and mathematical expressions, it is evident that the Q-function indicates how valuable an action a_t is at state s_t , while the V-function indicates the value of being in state s_t . Another important proxy function is the advantage function, defined as $A_{\pi}(s_t, a_t) := Q_{\pi}(s_t, a_t) - V_{\pi}(s_t)$ [25]. This function quantifies how much better or worse an action a_t is compared to the average action at state s_t .

By expanding the definitions and expressing the summation terms using Q or V , we can derive two recursive equations as

$$Q^{\pi}(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p(s_{t+1}|s_t, a_t)} [V^{\pi}(s_{t+1})],$$

$$V^{\pi}(s_t) = \mathbb{E}_{a_t \sim \pi(a_t|s_t)} [Q^{\pi}(s_t, a_t)].$$

By substituting these two equations into each other, we can further derive the Bellman equations [38] as

$$Q^{\pi}(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p(s_{t+1}|s_t, a_t), a_{t+1} \sim \pi(a_{t+1}|s_{t+1})} [Q^{\pi}(s_{t+1}, a_{t+1})],$$

$$V^{\pi}(s_t) = \mathbb{E}_{a_t \sim \pi(a_t|s_t)} [r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p(s_{t+1}|s_t, a_t)} [V^{\pi}(s_{t+1})]].$$

There are two primary methods for estimating value functions. For clarity, we focus on the V-function here, as the estimation of the Q-function can be similarly derived. The most straightforward approach is to use Monte Carlo (MC) estimation [85]. By performing multiple Monte Carlo samplings, we can approximate the value function as: $V(s_t) \approx \frac{1}{K} \sum_{k=1}^K \sum_{i=t}^{\infty} \gamma^{i-t} r(s_i^k, a_i^k)$. According to the law of large numbers [1], this method provides an unbiased estimation of the value function. However, it can be highly inefficient, costly, and time-consuming, especially when the agent is an LLM.

A more preferable approach is to use Temporal Difference (TD) learning [87]. The value Bellman equation, which resembles dynamic programming, bootstraps the value function over itself at the next timestep. Therefore, we can perform Bellman backups to estimate the value function. In the context of deep learning, we typically construct a value function (also known as a critic network) [56] and train it by minimizing the empirical Bellman error as

$$L(\phi) = \frac{1}{|\mathcal{D}|} \sum_{(o_t, a_t, o_{t+1}, r_t) \sim \mathcal{D}} [r_t(o_t, a_t) + \gamma V_{\phi}(o_{t+1}) - V_{\phi}(o_t)]^2.$$

Once the value function is obtained, the next crucial step is policy extraction, which involves deriving the policy from the value function [99]. The specific policy extracted can vary widely depending on the chosen method. For instance, a greedy policy can be extracted using $\pi(a_t|s_t) = \delta(a_t = \arg\max_a Q(s_t, a_t))$, where δ denotes the Dirac delta function. Alternatively, a soft policy can be derived as $\pi_{\text{MaxEnt}}(a_t|s_t) = \exp(\frac{1}{\alpha} (Q_{\text{soft}}(s_t, a_t) - V_{\text{soft}}(s_t)))$, where α is a temperature parameter [26]. Other extraction methods can yield different policies, including mixture policies. Algorithm 1 depicts the fundamental procedure of value-based deep RL methods. However, policy extraction in the context of LLMs can be highly counterintuitive. We will elaborate on the reasons for this in Section 3.3.

Algorithm 1: General value-based deep RL method

Input: Parameters ϕ_0 for value function Q , some policy derived from value function

e.g. ϵ -greedy policy $\pi_0(a|s) = \epsilon U(a) + (1 - \epsilon)\delta(a = \arg\max_a Q_{\phi_0}(s, a))$, some initial state distribution $d(s_0)$, transition probability $p(s'|s, a)$, discount factor γ , learning rate α , etc.

Output: Optimal Q function Q^* with parameters ϕ^* , extracted optimal policy π^*

```

1 initialize parameters  $\phi_0$  for value function  $Q_{\phi}$ , target value function w/o gradient  $Q_{\text{targ}} \leftarrow Q_{\phi_0}$ ,
   some policy  $\pi_0$ , replay buffer  $\mathcal{D} \leftarrow \emptyset$ 
2 for iterations  $i = 0, \dots$  do
3   initialize  $s \leftarrow s_0 \sim d(s_0)$ 
4   while not terminal state do
5     select action using policy  $a \sim \pi_i(a|s)$ 
6     observe next state  $s' \sim p(s'|s, a)$ 
7     get reward  $r = R(s, a)$ 
8      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s, a, s', r)\}$ 
9   end
10  sample a batch  $\mathcal{B} = \{(s, a, s', r)\} \sim \mathcal{D}$ 
11  compute Bellman error  $\epsilon = \sum_{\mathcal{B}} (Q_{\phi_i}(s, a) - (r + \gamma \max_{a'} Q_{\text{targ}}(s', a')))$ 
12  update  $\phi$  by minimizing  $\epsilon$ :  $\phi_{i+1} \leftarrow \phi_i - \alpha \nabla \epsilon$ 
13  extract policy  $\pi_{i+1}$  based on  $Q_{\phi_{i+1}}$ 
14  periodically update target value function:  $Q_{\text{targ}} \leftarrow Q_{\phi_i}$ 
15 end

```

Policy-based Methods Another approach to optimizing policies in RL involves directly estimating the gradient of the policy and performing gradient ascent. This method is largely based on the policy gradient (PG) theorem [86]. Recall that the objective is to maximize the expected reward:

$J(\theta) = \mathbb{E}_{\tau \sim p_{\pi_\theta}(\tau)} [\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$.¹ According to the policy gradient theorem, the gradient of the expected reward $J(\theta)$ can be expressed as

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_{\pi_\theta}(\tau)} \left[\sum_{t=0}^{\infty} \Psi_t \nabla_\theta \log \pi_\theta(a_t | s_t) \right],$$

where, in the simplest case, $\Psi_t = \sum_{t'=t}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'})$. However, the cumulative discounted reward is often difficult to obtain directly, so Ψ_t can be approximated using other proxies, such as the Q-value $Q^\pi(s, a)$ or the advantage function $A^\pi(s_t, a_t)$ [74].

Given the gradient of J with respect to the policy parameters θ , we can do gradient ascent to update the parameters as (1). The most classic method is REINFORCE [104], which utilizes MC sampling to approximate the gradient as

$$\begin{aligned} \nabla_\theta J(\theta) &\approx \frac{1}{K} \sum_{k=1}^K \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t^k | s_t^k) G_t^k, \\ \theta &\leftarrow \theta + \alpha \nabla_\theta J(\theta). \end{aligned} \tag{1}$$

Algorithm 2 illustrates the general process of policy-based deep RL methods. And most of the time, people subtract the cumulative reward with a baseline $b(s_t)$ defined as an average over the sampled trajectories to reduce the variance [25].

As we can see, policy gradient does not require a value function anymore, instead, it uses the true rewards to optimize the policy directly, which is more stable compared with value-based methods, especially when the policy is an LLM. Moreover, many other variants have been proposed, *e.g.* REINFORCE++ [31], RLOO [41], etc., and they seem to work relatively well in LLMs.

Algorithm 2: General policy-based deep RL method

Input: initial parameters θ_0 for policy π_θ , initial state distribution $d(s_0)$, transition probability $p(s' | s, a)$, discount factor γ , learning rate α , etc.

Output: Optimal policy π^*

```

1 initialize parameters  $\theta_0$  for policy  $\pi_\theta$ , replay buffer  $\mathcal{D} \leftarrow \emptyset$ 
2 for iterations  $i = 0, \dots$  do
3   sample initial states  $\mathbf{s}_0 \sim d(s_0)$  and generate trajectories  $\tau = \{\tau^k\} = \{(s_0, a_0, \dots, s_H, a_H)^k\}$ 
   using current policy  $\pi_\theta$ 
4   compute cumulative returns  $G_t^k = \sum_{t'=t}^H \gamma^{t'-t} r_{t'}^k(s_{t'}^k, a_{t'}^k)$  (or some other proxy) for all
   timesteps  $t$ 
5   estimate policy gradient  $\nabla_\theta J(\theta) \approx \frac{1}{K} \sum_{k=1}^K \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t^k | s_t^k) G_t^k$ 
6   update policy by gradient ascent  $\theta \leftarrow \theta + \alpha \sum_{t=0}^T \nabla_\theta J(\theta)$ 
7 end
```

Actor-Critic Actor-Critic (AC) [85, 40, 16, 57] architecture is a hybrid way, incorporating both value-based methods and policy-based methods, to optimize the policy. While value-based methods extract policy poorly and policy-based methods lack direct estimations to values with a result of low efficiency, Actor-Critic inherits fine-grained credit assignment from value-based methods and optimization stability from policy-based methods. Actor-Critic algorithms train the value function and policy iteratively. For each iteration, it updates the value function based on TD error and optimizes the policy via the policy gradient method

One of the most suitable AC algorithms that adapts well to LLMs is proximal policy optimization (PPO) [73], which is developed upon Trust Region Policy Optimization (TRPO) [72]. TRPO gives

¹In other papers you may see people denote it $J(\pi_\theta)$. In fact, these two expressions are the same. You can view it as some reward J being a function of parameters θ or some reward J being a function of the policy π , which is parameterized by θ .

a surrogate objective function to maximize, together with a constraint on policy update steps as

$$\underset{\theta}{\text{maximize}} \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t(s_t, a_t) \right], \text{ s.t. } \hat{\mathbb{E}}_t [\text{KL} [\pi_{\theta_{\text{old}}}(\cdot|s_t) \|\pi_{\theta}(\cdot|s_t)]] \leq \delta,$$

where $\hat{A}_t$ is an estimator of the advantage function at timestep t , which is usually computed using GAE [74], and $\hat{\mathbb{E}}_t$ is the empirical average over a finite batch of samples. By the Lagrange Multiplier, it is equivalent to the unconstrained problem as

$$\underset{\theta}{\text{maximize}} \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t(s_t, a_t) - \beta \text{KL} [\pi_{\theta_{\text{old}}}(\cdot|s_t) \|\pi_{\theta}(\cdot|s_t)] \right].$$

TRPO also gives a monotonic improvement guarantee for general stochastic policies, but for the optimization objective above, it concerns second-order optimization, including Fisher Information Matrix (FIM) computation, which can be costly. Based on TRPO, PPO emerges as a first-order algorithm that also reproduces the monotonic improvement of TRPO. PPO proposes a clipped surrogate object as

$$\underset{\theta}{\text{maximize}} \hat{\mathbb{E}}_t \left[\min \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A^{\pi_{\theta_{\text{old}}}}(s, a), \text{clip} \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}, 1 \pm \epsilon \right) A^{\pi_{\theta_{\text{old}}}}(s, a) \right) \right].$$

And countless experiments have shown that PPO is one of the most effective RL methods for nearly all decision-making tasks, though it still suffers from inefficiency and long training time due to the on-policy method and the nature of trial-and-error [58, 63, 28].

In LLMs, we can easily analogize from typical RL tasks, *e.g.*, mujoco [92], to language tasks. For example, an LLM is trying to complete the query a user gives, and the LLM generates actions once it receives observations, which can be the user’s further responses or function call results, also maybe together with a reward signal. Based on the interaction, or in RL, *i.e.*, trajectory, PPO or other methods can be utilized to reinforce the LLM to learn how to better tackle human queries in interactions [123, 23].

2.1.3 From RL to RFT

Table 1: Differences between RL and RFT.

	RL	RFT
Actor	Policy from scratch	Large-scale pretrained foundation model
Constraint	No constraint	Maintain original capabilities
Action Space Size	Typically small, numerable	Huge, $O(\mathcal{V}^{ L })$ or $O(\mathcal{V} \times L)$
Transition	Stochastic	Deterministic and Stochastic
Granularity	Action	Action or token
Process	Reset state and sample traj.	Reset query, sample traj. and verify answer
Sample Magnitude	Large	Relatively small and high-quality

Differences Though RL and RFT share foundational methodological principles, there still exist nuanced differences as listed in Table 1. RL, closely rooted in optimal control frameworks, typically trains policy agents, such as robots, from scratch, requiring no prior domain knowledge. In contrast, RFT operates on large-scale pretrained models and adapts these models to specialized tasks. Additionally, RL trains a policy specifically for a particular task within a specific environment, while RFT enhances a model’s task-specific capabilities without degrading its original abilities, such as language understanding or generative skills in LLMs. The environments also differ significantly. Conventional RL environments are entirely stochastic, whereas typical RFT agentic environments for LLMs can integrate both deterministic and stochastic transitions—deterministic for operations like sentence or token concatenation, and stochastic for environment feedback, such as tool execution

results or next user queries. Since LLMs generate tokens to form sentences, RFT can operate at both the action-level (sentence-level) and token-level, whereas traditional RL settings are typically action-level. Another major difference is that RL is known for its low sample efficiency, while RFT requires relatively fewer, high-quality samples.

Implementations Since we know the difference between RFT and RL, we can introduce lots of fine-tuning techniques to better show how RFT works and what is reflected in many existing research. The first is LoRA, which freezes the base model and injects trainable rank decomposition matrices into the model. LoRA is vital for fine-tuning LLMs, especially for LaMAS, because it can optimize the policy efficiently with minimal parameter updates and also makes LLM-based multi-agent training affordable and ideal. As a single model can have different LoRA adapters for different tasks, multi-agent systems can be made using only one base model equipped with multiple adapters [2]. Secondly, it becomes necessary to implement divergence constraints during training to prevent the policy update from straying too far from the initial or reference policy, which possesses the pretrained abilities. PPO and other trust-region-related algorithms provide strong guarantees for monotonic improvement, and in their objectives, we can also modify the KL divergence from between π_{old} to between π_{ref} as GRPO [76] does in (2).

Prevailing Objective Functions To further elaborate on the objective functions used in RFT, we will introduce several prominent methods, including GRPO, DAPO, and VAPO. These objective functions are designed to optimize the policy updates while ensuring stability and efficiency during the fine-tuning process.

GRPO utilizes the objective function as shown in (2) [76]. As stated in GRPO, q, o are questions and outputs sampled from the question dataset and the old policy π_{old} . This approach dispenses with the need for value function approximation, opting instead to compute a baseline through the average rewards of multiple sampled trajectories. Regularization is achieved by directly incorporating the KL divergence between the trained policy and the reference policy into the loss function.

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(O|q)]} \left\{ \frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip} \left(r_{i,t}(\theta), 1 \pm \epsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} \parallel \pi_{\text{ref}}] \right\} \right\}, \quad (2)$$

$$\text{where } r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})}, \quad \hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^G)}{\text{std}(\{R_i\}_{i=1}^G)}.$$

DAPO advances upon GRPO and adopts the objective function as shown in (3) [114]. It promotes exploration by increasing the ceiling clip parameter ϵ_{high} . DAPO further enhances its sampling strategy through dynamic sampling, which involves over-sampling and filtering out prompts with accuracy values of 1 and 0, ensuring that all prompts in the batch provide effective gradients. Additionally, DAPO implements a token-level policy gradient loss in the long-CoT RL scenario.

$$J_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\text{old}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip} \left(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}} \right) \hat{A}_{i,t} \right) \right],$$

$$\text{s.t. } 0 < \left| \left\{ o_i | \text{is_equivalent}(a, o_i) \right\} \right| < G. \quad (3)$$

VAPO extends DAPO and employs the objective function presented in (4) [115]. It further enhances the training process by incorporating an additional negative log-likelihood (NLL) loss for correct outcomes sampled during training. This additional loss term improves the utilization efficiency of

positive samples, thereby optimizing the overall learning performance.

$$L(\theta) = - \frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip} \left(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}} \right) \hat{A}_{i,t} \right) + \mu * L_{\text{NLL}}(\theta),$$

$$\text{where } L_{\text{NLL}}(\theta) = - \frac{\sum_{o_i \in \mathcal{T}} \sum_{t=1}^{|o_i|} \log \pi_{\theta}(a_t | s_t)}{\sum_{o_i \in \mathcal{T}} |o_i|}.$$
(4)

2.2 LLM-based Multi-Agent Systems (LaMAS)

The concept of LaMAS has recently gained prominence [108], with numerous applications and inference frameworks emerging to address complex agentic problems [17]. Unlike traditional MARL, where agents act synchronously with uniform decision weights, LaMAS introduces hierarchical organization and asynchronous execution. Agents in LaMAS can dynamically decompose tasks, adapt workflows based on execution dependencies, and coordinate actions in a decentralized yet goal-aligned manner.

Existing optimization methods of LaMAS can be categorized into two types as below:

- **Tuning-free techniques.** A prevalent optimization strategy for LaMAS involves tuning-free methods that do not update agent parameters. Examples include prompt engineering [19], in-context learning [91], and self-evolution [109, 33]. These techniques rely on iterative refinements of agent profiles or prompts to enhance task-specific performance.
- **Parameter fine-tuning.** A more sophisticated approach involves updating agent parameters or network configurations to optimize performance. This paradigm integrates deep learning and specific methodologies, such as multi-agent debating to generate high-quality training datasets [81] or fine-tuning programming modules for improved workflow organization [37]. CORY [49] duplicates the LLM into two agents, i.e., a pioneer and an observer, that interact through knowledge transfer and role exchange, improving training stability and performance. However, many parameter fine-tuning methods lack a theoretical foundation in RL, relying instead on ad-hoc techniques. SiriuS [120], a self-improving, multi-agent optimization framework using bootstrapped reasoning, iteratively refines agent behaviors by leveraging successful reasoning trajectories and augmenting unsuccessful ones. MARTI [118] develops an RL-oriented framework for multi-agent LLM training and inference, but still treats agents largely through separate policy trainers and full-parameter tuning, which can be costly for scalable LaMAS optimization. The most related work, MAPoRL [64], rooted in MARL, pioneers in applying multi-agent PPO to post-train LLMs, but does not consider the unique characteristics of LaMAS and assumes homogeneous agent behavior through collaborative debate, diverging from real-world scenarios where agents often perform diverse or orthogonal roles to solve complex problems. This article will focus on RFT and MARL-related methodologies in subsequent sections, providing a principled paradigm for general LaMAS reinforcement.

2.3 Multi-Agent Reinforcement Learning (MARL)

2.3.1 Decentralized Partially Observable Markov Decision Process

MARL is a variant of RL, which includes multiple agents acting **simultaneously** once receiving the environment observations, and it's often modeled by a certain form of MDP – *Decentralized POMDP* (*DEC-POMDP*) [60] $\langle \mathcal{N}, \mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma \rangle$, where $\mathcal{N} = \{1, \dots, n\}$ is the set of agents **without any order constraint**, $\mathcal{S}$ is the state space, $\mathcal{O} = \prod_{i=1}^n \mathcal{O}^i$ is the joint observation space, which is a product of local observation spaces at global state s , $\mathcal{A} = \prod_{i=1}^n \mathcal{A}^i$ is the joint action space, which is similarly a product of local action spaces, and for the others, we can borrow the explanation from Section 2.1.1.

2.3.2 Main Approaches of MARL

Independent Learning One of the most intuitive approaches to implementing MARL is through independent learning. In this setting, each agent operates as an independent entity, possessing its own policy and value function. Other agents are treated as part of the environment, and there is no direct communication between agents. Consequently, each agent optimizes its actions based solely on its own observations and rewards, without considering the policies of other agents.

This approach allows for the direct application of single-agent reinforcement learning methods to each agent. Examples include independent Q-learning [88, 78], independent policy gradient [14], and independent AC methods. These methods are straightforward to implement and can be easily scaled to various domains. However, independent learning has significant drawbacks. It can lead to instability and difficulty in converging to an optimal solution. For instance, some agents may develop selfish policies or become "lazy" in collaborative tasks, resulting in what is known as a social dilemma.

Early experiments have demonstrated that, for certain tasks, cooperative agents outperform independent agents, especially when communication is absent [88]. The cost of communication and the optimization of information exchange strategies require careful consideration in collaborative learning. Despite the challenges of instability and poor convergence, independent learning remains a fundamental and viable approach to MARL.

Centralized Training with Decentralized Execution Apart from independent learning, another way to MARL is by centralized training. During training, agents share a joint global value function $Q^\pi(\mathbf{o}, a^1, \dots, a^n)$ which takes into account the environment state and actions of other agents. For its design, Q can either output the value for the joint action and marginalize out for a specific action, as in COMA [20], or directly output the value for action a^i by agent i , as in MADDPG [47]. The latter approach allows for arbitrary reward structures for separate agents. Once the value learning component is established, policy optimization can be performed using the policy gradient: $\nabla_{\theta^i} J(\theta^i) = \mathbb{E}_{s \sim p^\mu, a^i \sim \pi^i} [\nabla_{\theta^i} \log \pi^i(a^i | o^i) Q^\pi(\mathbf{o}, a^1, \dots, a^n)]$ to update the policies.

As mentioned in Section 2.1.2, PPO has consistently been one of the most effective RL methods. Fortunately, it also extends well to multi-agent scenarios, leading to Multi-Agent PPO (MAPPO) [113]. Then, the objective of MAPPO is derived as

$$\text{maximize}_{\theta} \sum_{i=1}^n \mathbb{E}_{s \sim p^{\pi_{\theta_{\text{old}}}}, a \sim \pi_{\theta_{\text{old}}}} \left[\min \left(\frac{\pi_{\theta}^i(a^i | s)}{\pi_{\theta_{\text{old}}}^i(a^i | s)} A^{\pi_{\theta_{\text{old}}}}(s, \mathbf{a}), \text{clip} \left(\frac{\pi_{\theta}^i(a^i | s)}{\pi_{\theta_{\text{old}}}^i(a^i | s)}, 1 \pm \epsilon \right) A^{\pi_{\theta_{\text{old}}}}(s, \mathbf{a}) \right) \right].$$

However, this approach has a strong assumption that all agents share parameters, which leads to identical action spaces among agents. In the context of LLMs, it is common for different LLMs to have distinct action spaces. For instance, different LLMs may possess unique vocabularies, and some LLMs might even have different input-output formats, such as LLMs designed for ranking tasks. Moreover, MAPPO does not provide a rigorous guarantee of monotonic improvement, as a local improvement in one agent's policy can potentially degrade the overall team reward. HATRPO and HAPPO addressed this issue by sequentially updating the policies [43].

Communication and Coordination Another intriguing approach to MARL involves directly learning the communication among agents. For instance, designing a communication module or gate module can help establish an effective explicit communication and coordination mechanism within the multi-agent system.

One seminal work in this area is CommNet [82], which introduced a novel trainable controller Φ that maps global state-views s to joint policies $\mathbf{a}$. In this mapping, each agent has access to the hidden states of other agents, thereby influencing its own policy. This approach can be seamlessly integrated with RL algorithms and ideally results in an optimal communication module.

In the context of LLM-based multi-agent systems, such communication modules can take the form of an "orchestra agent". This agent is specifically designed and trained to act as a task and information router, facilitating effective collaboration among the agents. Currently, a popular solution involves designing a well-structured multi-agent framework that includes a planner or orchestration agent to enhance collaboration. However, most of these attempts focus on improving performance during inference without incorporating further training.

Sequential Modeling Another intriguing perspective on understanding MARL is through sequential modeling (SM). The connection between MARL and SM is grounded in the Multi-Agent Advantage Decomposition Theorem [42] as below:

Theorem 1 (Multi-Agent Advantage Decomposition Theorem [42]). *For any predefined permutation of n agents, for any state $s \in \mathcal{S}$ and joint action $\mathbf{a} \in \mathcal{A}$, the following always holds:*

$$A^\pi(s, \mathbf{a}^{1:n}) = \sum_{m=1}^n A^\pi(s, \mathbf{a}^{1:m-1}, \mathbf{a}^m). \quad (5)$$

This theorem reveals that if an agent is aware of the actions taken by its predecessors, maximizing the agent's local advantage is equivalent to maximizing the joint advantage. Building on this insight, a new multi-agent sequential decision-making paradigm has been proposed, exemplified by the Multi-Agent Transformer (MAT) [101]. Within this paradigm, the approach to approximating the value function remains consistent with conventional MARL methods, specifically by minimizing the empirical Bellman error below² as

$$L(\phi) = \frac{1}{nT} \sum_{i=1}^n \sum_{t=0}^{T-1} [r_t(\mathbf{o}_t, \mathbf{a}_t) + \gamma V_{\bar{\phi}}(\hat{\mathbf{o}}_{t+1}^i) - V_\phi(\hat{\mathbf{o}}_t^i)]^2,$$

where $\bar{\phi}$ is the parameters of the target network, which are non-differentiable and updated at intervals. But for the policy optimization, the modified clipping PPO objective goes as

$$L(\theta) = \frac{1}{nT} \sum_{i=1}^n \sum_{t=0}^{T-1} \min [\mathbf{r}_t^i(\theta) \hat{A}_t, \text{clip}(\mathbf{r}_t^i(\theta), 1 \pm \epsilon) \hat{A}_t], \text{ where } \mathbf{r}_t^i(\theta) = \frac{\pi_\theta^i(a_t^i | \hat{\mathbf{o}}_t^{1:n}, \hat{\mathbf{a}}_t^{1:i-1})}{\pi_{\theta_{\text{old}}}^i(a_t^i | \hat{\mathbf{o}}_t^{1:n}, \hat{\mathbf{a}}_t^{1:i-1})}.$$

An intuitive way to understand this paradigm is to consider that, before acting **simultaneously**, agents make decisions (not take actions) sequentially in an arbitrary order or permutation, based on what their predecessors have planned or decided to do. They then execute their actions accordingly. For instance, in a football game, it is akin to players stating their intended actions one by one during halftime. Once on the field, they follow through with the sequential decisions they have just made.

Networked Agents One more novel cooperative MARL paradigm involves networked agents, which operate in fully decentralized settings. In this framework, agents perceive local environmental observations and synchronize information locally with neighboring agents over a network [117]. A consensus mechanism has been introduced to the paradigm, expanding its applicability to real-world domains with strict privacy requirements [10, 95]. By enabling localized information sharing and coordination, networked agents enhance system robustness and adaptability while preserving data privacy, making this paradigm particularly promising for applications in decentralized and privacy-sensitive environments, especially for LaMAS in use.

3 MARFT Overview

Based on the review of RFT, LaMAS, and MARL, in this section, we will extend this foundation to the context of MARFT, highlighting the problem statement, unique considerations, and additional complexities that arise in this domain.

²In MAT, the *hat* $\hat{\cdot}$ means some encoding of the variable.

3.1 Problem Statement and Flex-MG

MARFT integrates the three key principles comprehensively reviewed in Section 2, adapting them to the context of RFT for LaMAS. In MARFT, agents operate within a framework that allows asynchronous yet coordinated execution, reflecting the dependencies inherent in agentic problem-solving (as detailed in Section 2.2). This design is particularly suitable for addressing complex tasks where execution dependencies necessitate dynamic adaptation and partial observability. Thus, the algorithms developed for MARFT serve dual objectives: optimizing concurrent actions and learning systemic organization and dynamic task decomposition workflow.

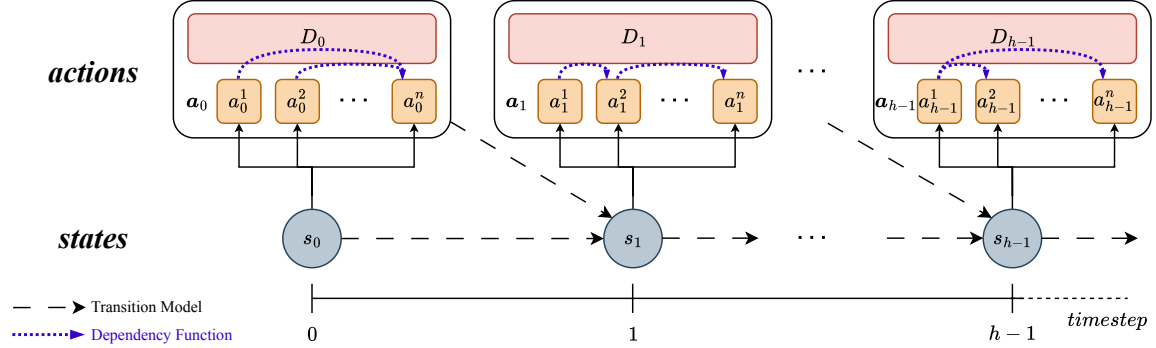


Figure 2: A detailed illustration of the dynamics of a Flex-MG. The dependency function (dashed purple line) can vary across timesteps.

To enhance problem formulation and accommodate MARFT, we propose *Flexible Markov Game (Flex-MG)*, illustrated in Figure 2 and denoted as $\langle \mathcal{V}, \mathcal{N}, \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma, \mathcal{D} \rangle$. Here, $\mathcal{V}$ is the vocabulary, $\mathcal{N} = \{1, \dots, n\}$ is the agent composition of a LaMAS **with some organization**, i.e., some order constraint among agents, $\mathcal{S}$ is the state space, and $\mathcal{A} = \prod_{i=1}^n \mathcal{A}^i$ is the joint action space. The transition function $\mathcal{T}: \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}'$, written as $P(s'|s, \mathbf{a})$, models state changes given joint actions. The reward function $\mathcal{R}: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, denoted $R(s, \mathbf{a})$, assigns rewards based on states and joint actions. The discount factor γ accounts for temporal reward decay.

Flex-MG is distinguished by its introduction of a **dependency function** $\mathcal{D}: \mathcal{A} \times \mathcal{A} \rightarrow \{0, 1\}$, which explicitly models inter-agent dependencies. Specifically, $\mathcal{D}(a^i, a^j) = 1$ indicates that the action of agent j is conditionally dependent on the action of agent i . As a result, the decision-making process of agent j is influenced not only by the global state s but also by the actions of all agents k such that $k \in \{i | \mathcal{D}(a^i, a^j) = 1\}$. These dependencies can be integrated into agent j 's input via concatenation or other fusion methods, thereby forming an enriched input for the agent's policy. Crucially, the dependency function $\mathcal{D}$ can vary across timesteps, potentially governed by an orchestration agent or a dynamic routing mechanism. This flexibility captures the evolving coordination structures in LaMAS and embodies the "flexible" nature of the proposed framework. Notably, when $\mathcal{D}(a^i, a^j) = 0$ for all i, j , Flex-MG reduces to a standard multi-agent decentralized setting where agents act independently.

3.2 Categorizations of LaMAS in MARFT

In the context of MARFT, the unique characteristics of LaMAS necessitate a detailed categorization. We classify LaMAS based on three key dimensions: parameter sharing, execution synchronicity, and update scheme. This categorization not only captures the distinct features of LaMAS but also provides a clear direction for the development of MARFT algorithms.

Parameter sharing Parameter sharing is a critical consideration in multi-agent systems, particularly in traditional MARL. However, in LaMAS, parameter differentiation strategies introduce additional complexity. Agents may either share parameters or maintain distinct configurations, each with unique implications for optimization.

For parameter sharing, agents share an identical base model, which can be fully fine-tuned or LoRA fine-tuned on a single adapter for task-specific roles. For non-parameter sharing, agents may use different base models with fine-tuning (full or LoRA) or share a common base model with role-specific adapter modules. While parameter sharing streamlines optimization and alleviates memory pressure, it can introduce challenges such as opposite optimization directions or parameter drift, where conflicting updates degrade performance. Despite these risks, parameter sharing remains essential for efficient LaMAS fine-tuning, particularly when combined with techniques like LoRA, which isolate task-specific adaptations from the base model.

Execution synchronicity Execution synchronicity represents another critical dimension for classifying LaMAS. Unlike traditional cooperative games in multi-agent systems, where agents typically act synchronously, LaMAS often operates under asynchronous or hybrid execution paradigms. This distinction arises from the inherent dependencies in agentic tasks, where execution outcomes frequently rely on partial or sequential results.

In synchronous LaMAS, agents execute actions concurrently, assuming full observability and independence of execution results, while in asynchronous LaMAS, agents act independently, with execution triggered by task-specific dependencies or environmental signals. The latter paradigm is prevalent in real-world applications, as it accommodates dynamic workflows and partial observability. Hybrid LaMAS combines synchronous and asynchronous elements to balance efficiency and flexibility.

Update simultaneity In addition to execution synchronicity during inference, the choice of update scheme during training further distinguishes LaMAS configurations. Specifically, LaMAS can be categorized into two types based on how agent updates are orchestrated.

In a LaMAS with synchronous update manner, all agents are updated concurrently. Though simple and easy to implement, it introduces non-stationarity into the optimization process and suffers from severe off-policy issues as later agents may base their updates on outdated policies of earlier agents. Each agent’s policy updates are based on rapidly changing system dynamics, complicating convergence and potentially leading to oscillations in performance. On the other hand, in sequential (i.e., agent-by-agent) update LaMAS, agents are updated sequentially, with each agent’s update based on the most current policies of others. This method ensures monotonic improvement of both individual agents and the overall system by reducing non-stationarity and minimizing off-policy effects. However, it may increase computational overhead due to the sequential nature of updates.

3.3 Differences and Resulting Challenges

Due to the distinct characteristics of LaMAS compared to traditional multi-agent systems in MARL, implementing large-scale LaMAS within a MARL framework presents significant challenges, and classic MARL methods often fail to effectively enhance the performance of LaMAS.³ Below, we list the key differences between MARFT and traditional MARL in table 2, and expand the discussion in detail, which hopefully can guide the methodology and implementation in the following section.

Execution Asynchronicity One of the most significant differences between conventional MARL and MARFT is the nature of agent action executions. In traditional cooperative MARL, agents’ actions are typically executed simultaneously. However, in MARFT, they are executed asynchronously. In some cases, the actions of certain agents may even depend on the outcomes of other agents’ actions. For example, in a collaborative coding assistant system, one LLM agent generates a function prototype, and another agent asynchronously refines it based on the first agent’s output, demonstrating both asynchronicity and result dependency. As a result, conventional MARL methods, which assume synchronous actions, may not be directly applicable or effective in LLM-based multi-agent systems.

³In fact, there is very little research about implementing classic MARL methods, such as COMA, MADDPG, MAPPO, HAPPO, etc., to LaMAS.

Table 2: Differences between MARFT and traditional MARL.

	MARL	MARFT
Execution Asynchronicity	Synchronous	Asynchronous
Utilities	Task-specific only	Agentic with original language ability
Agent Identity	Unspecified, usually id	Specified system prompts as profiles
Heterogeneity	Parameters	In-out formats, externals, parameters
System Organization	Static	Static/Dynamic
Optimization Space	Small	Large and variable

Utilities Unlike agents in typical MARL problems, LLMs are initially designed for language processing rather than specific agentic tasks. However, the value- or reward-guided nature of RL means that when using value-based RL algorithms to optimize or extract policies, the derived optimal policy often prioritizes actions that maximize the value function. This can lead to policies that generate high-reward actions or tokens but are incomprehensible to humans. Though some attempts to extract a policy together with entropy regularization have been made to improve its agentic intelligence without harming the text capability [102], it is still a point that requires additional attention when we try to implement value-based methods to optimize LLMs.

Characteristic Profiles The transition from single-agent to multi-agent systems in typical MARL benchmarks, such as Multi-Agent MuJoCo, often involves splitting a single agent into multiple components without additional modifications. In contrast, LLMs require a more nuanced approach. When decomposing tasks into orthogonal sub-tasks, each LLM agent needs a profile to define its role and capabilities, enabling it to generate actions consistent with its assigned character. This profile can be human-specified or learned by the agent through natural evolution. Consequently, the joint observation space in MARFT is augmented with profiles, taking the form [agent profile + env observation (+ agent-specific guidance)]. This modification is crucial when designing a MARL training framework.

Heterogeneity In conventional MARL benchmarks, heterogeneity is typically characterized by differences in agent structures and non-parameter-sharing schemes. However, LLM-based systems introduce a higher level of complexity. First, LLMs themselves can vary significantly in terms of model structures, parameters, input and output formats (*e.g.*, LLMs vs. Vision-Language Models), and vocabularies. Second, agents may have access to different external tools or devices, reflecting real-world scenarios. For instance, in designing a multi-agent system for a mobile operating system, some agents may have access to both local and remote search engines, while others may rely on proprietary databases or tools from other companies. This heterogeneity complicates training, making it more difficult and unstable.

System Organization Traditional MARL tasks are often set in static simulated environments. In contrast, LLM-based multi-agent systems are designed for real-life agentic tasks with higher uncertainty. These tasks can be decomposed in various ways, leading to different multi-agent systems with distinct populations and roles. Moreover, agents may exhibit sequential dependencies and contextual relationships when solving sub-tasks. For example, one agent’s action may depend on the outcome of another agent’s action. This dynamic organization can be either human-designed or agent-explored (*e.g.*, through training), adding another layer of complexity to the design process.

Optimization Space If we take one LLM generation as an action, the observation space is exponentially vast as $o \in \mathcal{O} \subseteq \mathcal{V}^L$, whose complexity is $O(|\mathcal{V}|^L)$, where $\mathcal{V}$ is the vocabulary of the acting LLM and N is the token length of the generated action, and both of them vary with different acting LLMs. It makes the value function hard to converge with stability. If we take one token as an action, the complexity can be reduced to $O(|\mathcal{V}| \times L)$, but it will become a task with super sparse reward outcome so that the value function should also be meticulously learned, as L can be

extremely large. This is also reflected in VAPO and is mitigated by value pretraining [115]. Fortunately, not all LLM-related tasks face such large optimization spaces. For instance, in embodied AI with LLMs, predefined actions (*e.g.*, "go to the kitchen," "grab a coffee") are often provided, significantly narrowing the action space [8, 90].

These differences collectively make the optimization of LaMAS more complex than traditional MARL problems. MARFT is designed to tackle the difficulties. To generally optimize the LaMAS regardless of the highly dynamic workflow or organization, MARFT reframes the problem as a sequential decision-making problem despite the dynamic organization. It preserves pretrained utilities by applying clipping, preventing excessive policy drift. Agent profiles are encoded to activate specific capabilities, and no modality assumptions are made, as all agent actions are expressed in tokens from their respective vocabularies.

4 Multi-Agent Reinforcement Fine-Tuning

Building on the formulations outlined in the previous section, we now turn to the core methodologies that enable MARFT to address the unique challenges of optimizing LaMAS.

4.1 MARFT Instantiations

Inheriting the SM-style re-modeling via the multi-agent advantage decomposition theorem introduced in the last section, MARFT follows the procedure demonstrated in Figure 3. Given any organizational or task-solving workflow of a LaMAS, theorem (5) allows MARFT to reframe it as a sequential decision-making process. When collecting interactive trajectories, the LaMAS generates actions with an auto-regressive problem-solving style. The "encoder" constructs local roll-out states using agent profiles and other relevant information, while the "decoder" combines these states with dependent predecessors' actions to generate the current agent's action $a^m \sim p_{\pi^m}(a^m | s, \mathbf{a}^{1:m-1})$.

To instantiate the framework, we first present MARFT on the action level, a variant designed for targeted, action-level policy optimization in LaMAS. During training, MARFT adopts an optimization objective in (6)⁴. Notably, each agent optimizes within a trust region conditioned on predecessor actions, akin to MAT, thus ensuring monotonic improvement as in HAPPO's sequential update scheme.

$$L(\theta) = \frac{1}{nT} \sum_{i=1}^n \sum_{t=0}^{T-1} \min \left[\mathbf{r}_t^m(\theta) \hat{A}_t, \text{clip}(\mathbf{r}_t^m(\theta), 1 \pm \epsilon) \hat{A}_t \right], \quad (6)$$

$$\text{where } \mathbf{r}_t^m(\theta) = \frac{\pi_{\theta^m}^m(a_t^m | s_t, \hat{\mathbf{a}}_t^{1:m-1})}{\pi_{\theta_{\text{old}}^m}^m(a_t^m | s_t, \hat{\mathbf{a}}_t^{1:m-1})}.$$

Action Normalization. As shown in equation (7), the probability of each token $P(w|\cdot)$ is less than (or equal to) 1. As a result, longer action tends to have a lower joint probability, even though it is more reasonable than other actions in the task. To address the issue, token normalization and word normalization divide the joint action probability in (7) by a constant related to its token or word length [90]. This gets rid of the strong assumption that all the actions have similar lengths, which in LLMs can be an evil condition, because in agentic tasks, it is all too common that solutions generated can be either super long with a chain-of-thought or super short with a direct answer.

$$\begin{aligned} \log P_{\pi^i}(a^i | s, \mathbf{a}^{\mathcal{N}(i)}) &= \log P_{\pi^i}(w^{i,1}, \dots, w^{i,L^i} | s, \mathbf{a}^{\mathcal{N}(i)}) \\ &= \sum_{j=1}^{L^i} \log P_{\pi^i}(w^{i,j} | s, \mathbf{a}^{\mathcal{N}(i)}, w^{i,1}, \dots, w^{i,j-1}). \end{aligned} \quad (7)$$

⁴Our current MARFT instantiation employs a sequential decision-making process for scalability. Extending it to handle the parallel rollouts permitted by Flex-MG (*i.e.*, a general DAG structure) would necessitate a more complex optimization objective to tackle off-policy problems, which we leave as a direction for future work.

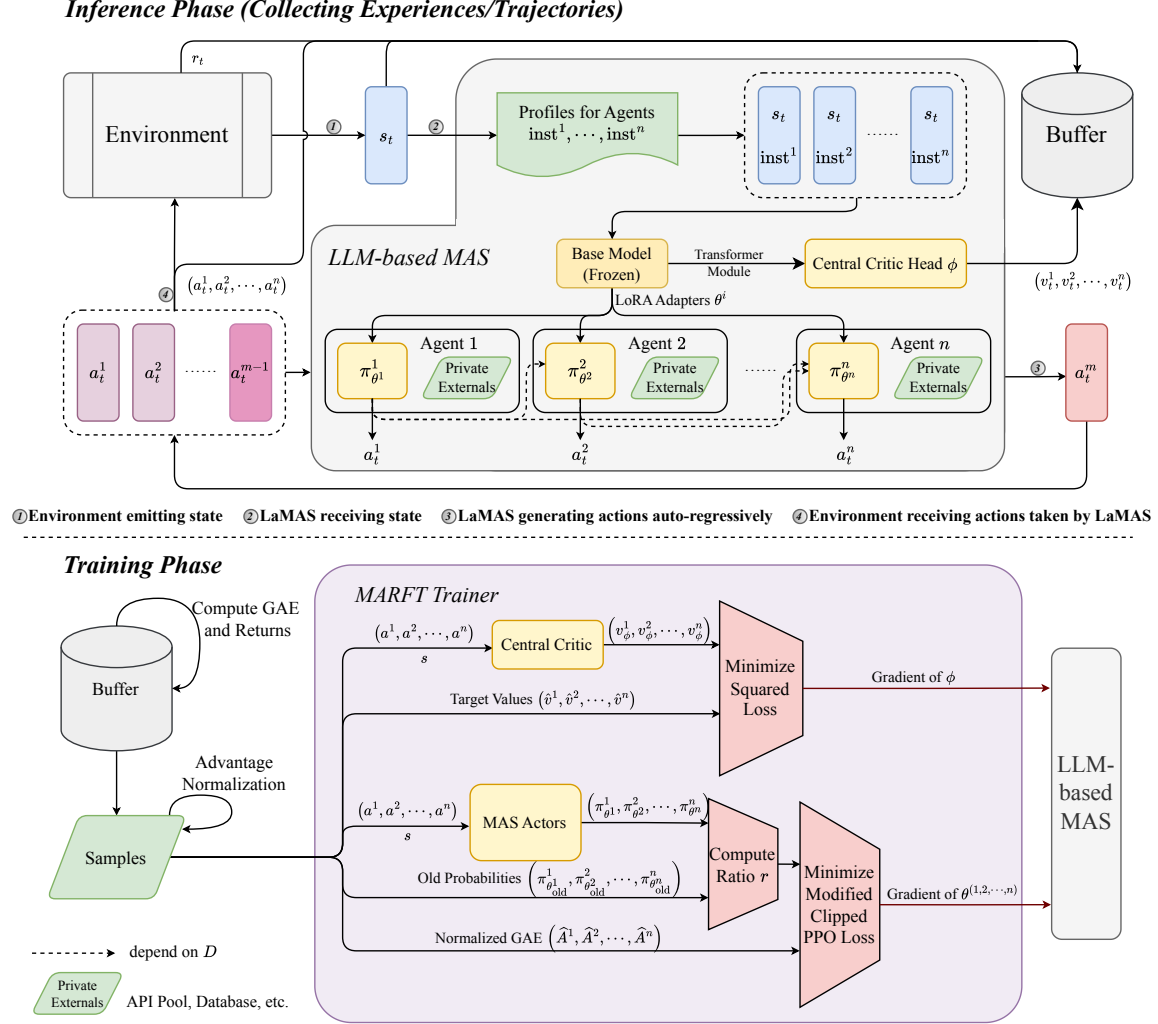


Figure 3: The procedure of Multi-Agent Reinforcement Fine-Tuning. Inference and training are conducted in an alternating manner. Each agent within the MAS can have its own private APIs, tool pool, database, and other resources.

Agent-by-agent Update. Updating every policy during each training epoch may introduce implicit off-policy issues, which can potentially compromise the monotonic improvement guarantee of PPO for individual agents' policies. This problem can be exacerbated in LaMAS, where agents often have dependencies on the results of other agents' actions. Drawing on the approach of A2PO [98], an agent-by-agent update scheme can be employed to restore monotonic improvement for each agent and facilitate stable convergence.

Trajectory-level Optimization. For language models, the definition of action from the RL perspective can be dynamic. Direct trajectory-level optimization, such as GRPO with outcome supervision [76] and its variant DAPO [114], can be viewed as an action-level optimization with non-zero reward only at the end of the trajectory in the MDP or a normal action-level optimization in a single-step variant of the MDP. Consequently, trajectory-level optimization can be seamlessly integrated into MARFT by following the same idea.

On top of the credit assignment of each agent on its action, we can also give a more fine-grained credit assignment by naturally treating every token as an action, leading to another token-level instantiation of MARFT. In this situation, we need to define the token-level multi-agent value functions as

$$V^\pi((s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,j-1})) \triangleq \mathbb{E}_{\tau \sim p_\pi(\tau|(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,j-1}))} \left[\sum_{k=t}^{\infty} \gamma^{k-t} R_k(s_k, \mathbf{a}_k^{1:n}) | (s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,j-1}) \right],$$

$$Q^\pi((s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,j-1}), w_t^{m,j}) \triangleq \mathbb{E}_{\tau \sim p_\pi(\tau|(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,j-1}), w_t^{m,j})} \left[\sum_{k=t}^{\infty} \gamma^{k-t} R_k(s_k, \mathbf{a}_k^{1:n}) | (s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,j-1}), w_t^{m,j} \right],$$

And then the token-level Bellman backup can be spontaneously derived as

$$V^\pi(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j}) \leftarrow \begin{cases} 0 + \gamma_w V^\pi(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j+1}) & \text{if } j < |a_t^m| \\ 0 + V^\pi(s_t, \mathbf{a}_t^{1:m}, w_t^{m+1,1}) & \text{if } j = |a_t^m| \text{ \& } m < n, \\ R(s_t, \mathbf{a}_t^{1:m}) + \gamma_a V^\pi(s_{t+1}, \mathbf{a}_{t+1}^{1:m}, \emptyset) & \text{if } j = |a_t^m| \text{ \& } m = n \end{cases}$$

$$Q^\pi(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j-1}, w_t^{m,j}) \leftarrow \begin{cases} 0 + \gamma_w \max_{w_t^{m,j+1}} Q^\pi(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j}, w_t^{m,j+1}), & \text{if } j < |a_t^m| \\ 0 + \max_{w_t^{m+1,1}} Q^\pi(s_t, \mathbf{a}_t^{1:m}, w_t^{m+1,1}), & \text{if } j = |a_t^m| \text{ \& } m < n \\ R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a \max_{w_{t+1}^{1,1}} Q^\pi(s_{t+1}, w_{t+1}^{1,1}), & \text{if } j = |a_t^m| \text{ \& } m = n \end{cases}$$

By guiding TD error and advantage computations, modified token-level Bellman backups enable token-level policy optimization. Treating each token as an individual action introduces a new Markov Game (MG) where rewards are exclusively allocated to the multi-agent system's final token. While this extreme sparsity complicates value learning, it drastically reduces optimization complexity from $O(|\mathcal{V}|^L)$ to $O(|\mathcal{V}| \times L)$. However, this altered MG formulation may cause MARFT to optimize inconsistently with the original problem [103]. To maintain the same optimality of the token-level MARFT, given an optimal multi-agent system π^* after sufficient training following the token-level Bellman backups, the optimal value functions should satisfy $Q^{\pi^*}(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j-1}, w_t^{m,j}) = Q^{\pi^*}(s_t, \mathbf{a}_t^{1:n})$ and $V^{\pi^*}(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j}) = V^{\pi^*}(s_t)$ for $\forall m \leq n$ and $j \leq |a_t^m|$. To further illustrate the gap between action-level optimization and token-level optimization, we expand the Bellman backup process over each token starting from arbitrary $m \leq n$, and $j \leq |a_t^m|$ as the following equations:

$$Q^{\pi^*}(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j-1}, w_t^j) = \underbrace{R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a \max_{\mathbf{a}_{t+1}^{1:n}} Q^{\pi^*}(s_{t+1}, \mathbf{a}_{t+1}^{1:n})}_{Q^{\pi^*}(s_t, \mathbf{a}_t^{1:n})} - \underbrace{\left[(1 - \gamma_w^{E_1}) R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a (1 - \gamma_w^{E_2}) \max_{\mathbf{a}_{t+1}^{1:n}} Q^{\pi^*}(s_{t+1}, \mathbf{a}_{t+1}^{1:n}) \right]}_{\text{The discrepancy}},$$

$$V^{\pi^*}(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j}) = \underbrace{R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a V^{\pi^*}(s_{t+1})}_{V^{\pi^*}(s_t)} - \underbrace{\left[(1 - \gamma_w^{E_1}) R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a (1 - \gamma_w^{E_1}) V^{\pi^*}(s_{t+1}) \right]}_{\text{The discrepancy}},$$

where $E_1 = \sum_{k=m}^n |a_t^k| - j - (n - m)$, $E_2 = \sum_{k=m}^n |a_t^k| + \sum_{k=m}^n |a_{t+1}^k| + m - j - 2n$, and $E_1, E_2 \neq 0$. (For proof, please see Appendix A.) The equations above, derived by expanding the Bellman backup, reveal the conditionally inconsistent optimality between action-level optimization and token-level optimization and give us instructions about how to deal with the intra-action discount:

1. As the intra-action discount γ_w decreases, the discrepancy is enlarged.
2. Only when $\gamma_w = 1$, the action-level optimization and token-level optimization achieve consistent optimality.

4.2 Implementations

4.2.1 Base Algorithm

The detailed algorithm implementation of MARFT is illustrated in Algorithm 3. For the design of the value function V_ϕ , we base it on a frozen transformer module, which processes the sequence information and outputs the final hidden state vector of the sequence, and add an extra trainable multilayer perceptron (MLP) with parameters ϕ to map the hidden vector to a specific value. A common instantiation of the transformer module can be the base model of an agent. For the design of the agents, we usually resort to LoRA fine-tuning to avoid overwhelming pressure on computing resources. If the agents share the same base model, we can assign separate but same-structured LoRA adapters to them, but if the agents have different base models, even different modalities, we then have to give them distinguished LoRA adapters.

Algorithm 3: Multi-Agent Reinforcement Fine-Tuning (MARFT)

Input: Agent population n , agent profiles inst^i , the initial joint policy π_{θ_0} with parameters θ_0^i for each policy $\pi_{\theta^i}^i$, initial parameters ϕ_0 for the critic network V_ϕ , hyper-parameters including maximum interaction steps T in one roll-out, clip parameter ϵ , discount factor γ , GAE λ , etc.

Output: Optimized joint policy π and critic network ϕ

```

1 initialize policy  $\pi_\theta \leftarrow \pi_{\theta_0}$ , critic network  $V_\phi \leftarrow V_{\phi_0}$  and buffer  $\mathcal{D} \leftarrow \emptyset$ 
2 for  $episode = 0, \dots$  do
3   for  $t = 0, \dots, T - 1$  do
4     collect  $s_t$ 
5     for  $i = 1, \dots, n$  do
6       generate action  $a^i \sim p_{\pi_{\theta^i}^i}(a^i | s, \mathbf{a}^{1:i-1})$ 
7     end
8      $s_{t+1} \sim \mathcal{T}(\cdot | s_t, \mathbf{a}_t)$ 
9      $r_t = R(s_t, \mathbf{a}_t)$ 
10     $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, \mathbf{a}_t, r_t, s_{t+1})\}$ 
11  end
12  compute advantage estimate  $\hat{A}$  via GAE and compute value network target  $\hat{V}$  in  $\mathcal{D}$ 
13  for  $n \text{ epochs}$  do
14    sample a batch  $\mathcal{B} = \{(s_t, \mathbf{a}_t, r_t, s_{t+1}, \hat{A}, \hat{V})\} \sim \mathcal{D}$ 
15    update  $\phi$  by minimizing  $\sum_{n=1}^N \|V_\phi(s_n) - \hat{V}_n\|^2$ 
16    update  $\theta$  by maximizing the objective (6)
17  end
18 end

```

For the implementation of action normalization, we can directly count the token length or word length as Z and divide the sum of the log probabilities of generated tokens by Z .

$$\text{Norm} \left(\log P_{\pi_{\theta^i}^i}(a^i | s, \mathbf{a}^{1:i-1}) \right) = \frac{\log P_{\pi_{\theta^i}^i}(a^i | s, \mathbf{a}^{1:i-1})}{Z}.$$

To implement agent-by-agent update, this sequential update mechanism involves updating only one policy per training epoch and cycling through the policies alternatively.

4.2.2 Token-level Adaptations

For the implementation of Token-level MARFT, the overall procedure mirrors that of Algorithm 3, with the primary modification being the storage of all token logits and values, rather than just the joint action probabilities and values, during the trajectory rollout phase. Subsequently, the token-level Bellman backup is utilized to determine the target values for the critic network and to calculate

the TD errors. Additionally, GAE [74] is employed to estimate the advantage values for each token. The optimization objectives remain consistent with those outlined in the general framework.

5 Experiments

To better demonstrate the effectiveness and superiority of the MARFT paradigm, we conduct comprehensive experiments. We first compare MARFT against the representative LoRA fine-tuning method MAPoRL [64], which is essentially Independent PPO (IPPO), to demonstrate the stability and performance advantages of MARFT. We then conduct ablation studies on LaMAS role specification. Our implementation is built upon the open-source AReaL codebase [21].

5.1 Experiment Setups

Benchmarks and Reward. We conduct experiments on DeepScaler [89] and DeepCoder [4], respectively, for mathematical and coding tasks. After training in the two environments, we test the optimized LaMAS on various out-of-domain benchmarks, including LiveCodeBench [35] and CodeForces [66] for coding, and AIME2024 [50], MATH500 [29, 46], Minerva Math [45], DeepScaler [89], and OlympiadBench [27] for math. The reward in the math problem-solving environment is 1 for correct answers and 0 otherwise, whereas in the coding environment, it is the test-case pass rate.

Base Model and LaMAS Configurations. For base models, we utilize the Qwen2.5 series [107], training Qwen2.5-1.5B-Instruct for math problems and Qwen2.5-3B-Instruct for coding. Additionally, we explore different LaMAS setups: DUO (Planner $\rightarrow$ Solver), TRIO (Planner $\rightarrow$ Solver $\rightarrow$ Verifier), and QUAD (Planner $\rightarrow$ Solver $\rightarrow$ Reflector $\rightarrow$ Verifier). Each role has a concise role description as its system prompt. The last agent gives the final answer.

LoRA Fine-Tuning Configurations. We utilize LoRA to fine-tune the models, set LoRA rank to 32 and LoRA alpha to 16, and set target modules to all-linear. Thus, a LoRA adapter only has about 2% tunable parameters of the base model. For a LaMAS consisting of n agents, MARFT tunes one LoRA adapter for agents when sharing policy, otherwise n LoRA adapters, and another adapter for the centralized critic. In contrast, Independent PPO tunes $2n$ LoRA adapters for n independent agents with n critics.

5.2 Comprehensive Comparison Results

5.2.1 MARFT outperforms Independent PPO (MAPoRL).

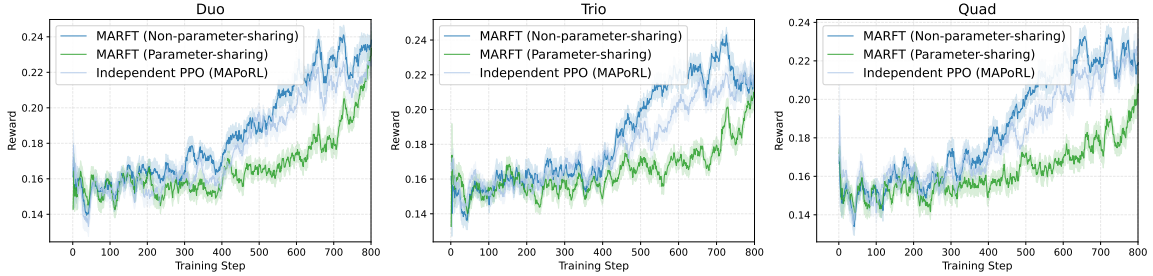
Table 3 shows the performance of vanilla LaMAS and fine-tuned ones on various evaluation benchmarks. For all LaMAS configurations, MARFT achieves higher scores on most benchmarks, and especially as the agent number increases, this superiority becomes larger. When the agent number grows to 4, MARFT outperforms IPPO (MAPoRL) on all benchmarks by a relatively large margin. We credit this to the centralized critic and the superior credit assignment across agents, which guides them to take actions that are not only good for their current state but also for the whole team’s reward.

5.2.2 MARFT provides monotonous improvement.

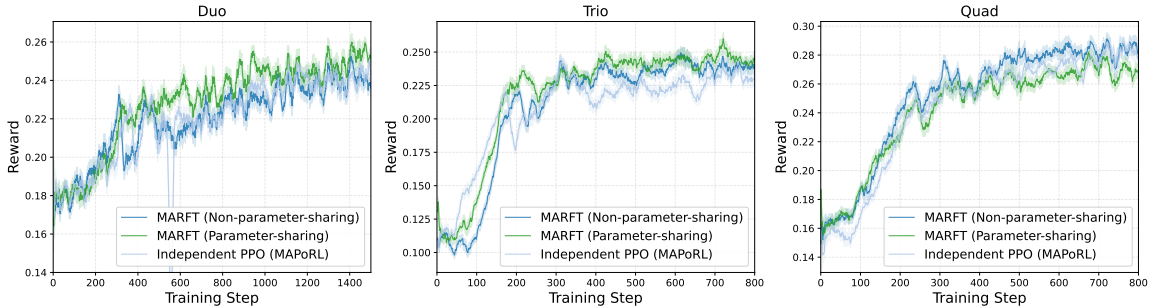
Figure 4 demonstrates the training dynamics of different LaMAS being trained using MARFT and IPPO. Across both mathematical and coding environments and all DUO, TRIO, and QUAD settings, MARFT consistently outperforms IPPO in convergence speed and final reward, indicating stronger collaborative optimization. IPPO also shows a small DUO crash around 550 training steps, whereas MARFT-tuned LaMAS improve steadily. These dynamics align with the monotonic-improvement guarantee supported by multi-agent advantage decomposition and trust-region learning.

Table 3: Performance comparison of MARFT and Independent PPO (MAPoRL) across different evaluation benchmarks. Metrics reported are accuracy for math problems and total pass rate for coding problems. All values represent the mean across 5 random seeds.

LaMAS	Benchmark	Vanilla	IPPO (MAPoRL)	MARFT (Ours)
DUO	DeepCoder	45.28	46.95 (+1.67)	48.20 (+2.92)
	LiveCodeBench	169.19	176.93 (+7.74)	176.44 (+7.25)
	CodeForces	24.04	24.28 (+0.24)	24.85 (+0.81)
	AIME2024	2.00	3.33 (+1.33)	3.33 (+1.33)
	MATH500	43.20	49.48 (+6.28)	49.48 (+6.28)
	Minerva Math	13.46	16.32 (+2.86)	16.25 (+2.79)
	DeepScaler	20.15	22.46 (+2.31)	22.47 (+2.32)
	OlympiadBench	16.08	18.31 (+2.23)	18.43 (+2.35)
TRIO	DeepCoder	50.38	51.38 (+1.00)	50.46 (+0.08)
	LiveCodeBench	160.51	162.72 (+2.21)	165.43 (+4.92)
	CodeForces	24.00	24.06 (+0.06)	25.51 (+1.51)
	AIME2024	2.00	4.00 (+2.00)	5.33 (+3.33)
	MATH500	45.76	49.12 (+3.36)	49.52 (+3.76)
	Minerva Math	15.81	16.76 (+0.95)	16.84 (+1.03)
	DeepScaler	20.79	22.54 (+1.75)	22.45 (+1.66)
	OlympiadBench	17.60	19.05 (+1.45)	18.72 (+1.12)
QUAD	DeepCoder	47.16	51.30 (+4.14)	52.67 (+5.51)
	LiveCodeBench	152.09	156.48 (+4.39)	160.64 (+8.55)
	CodeForces	20.50	21.88 (+1.38)	25.07 (+4.57)
	AIME2024	2.00	2.67 (+0.67)	3.33 (+1.33)
	MATH500	46.64	49.76 (+3.12)	49.84 (+3.20)
	Minerva Math	15.29	15.81 (+0.52)	16.99 (+1.70)
	DeepScaler	21.23	22.24 (+1.01)	22.36 (+1.13)
	OlympiadBench	16.60	18.37 (+1.77)	18.55 (+1.95)



(a) Training reward on DeepScaler.



(b) Training reward on DeepCoder.

Figure 4: Training rewards when fine-tuning LaMAS using MARFT and IPPO across different environments and agent scales.

An interesting phenomenon is that from both the evaluation table and the training reward curves, we do not see a substantial improvement brought by scaling the agent number. We attribute this performance saturation to the strong diminishing returns inherent in scaling homogeneous multi-agent systems. Recent information-theoretic analyses reveal that multi-agent performance is fundamentally bounded by intrinsic task uncertainty rather than raw agent count [112]. Instead, actual information gain depends strictly on the number of effective channels, i.e., independent, non-redundant reasoning paths generated by the agents. In our current DUO, TRIO, and QUAD configurations, the agents share identical base models and operate as a homogeneous ensemble. Consequently, scaling the agent count from 2 to 4 primarily produces highly correlated trajectories, injecting redundant evidence into the system rather than complementary information.

Crucially, however, while naive scaling yields marginal information gain due to this homogeneity, MARFT consistently elevates the absolute capabilities of LaMAS at every scale. Although the ensemble’s effective channel count remains saturated, MARFT fundamentally improves how these channels are utilized through rigorous token-level optimization. This demonstrates that even when scale-driven diversity is bottlenecked, principled multi-agent reinforcement fine-tuning remains a robust and indispensable mechanism for enhancing LLM-based MAS capabilities.

5.3 Self-Evolution: Ablations on Role Specification

What happens if we set the LaMAS purely blank? This means we do not specify role descriptions as system prompts, i.e., agents are anonymous; instead, we let the LaMAS find the division of roles. We conduct an ablation experiment on LaMAS specification, and Figure 5 shows the reward dynamics of the pre-configured and anonymous settings during MARFT. Surprisingly, in all comparison experiments, the anonymous setting dominates the pre-configured one. This phenomenon indicates that when initialized by general base models, intentionally specifying agent roles harms optimization in the early and middle stages, because agents need to first adapt to the roles they are assigned to.

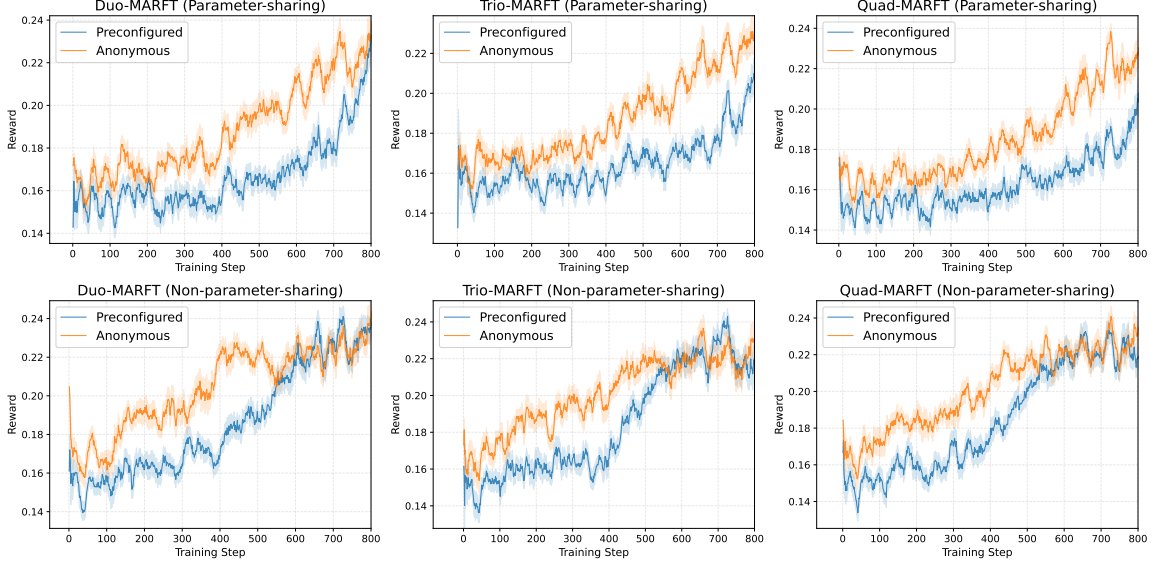


Figure 5: Training rewards on DeepScaler when LaMAS specifications are different.

6 Perspectives

From this section onward, we shift our focus from the technical aspects of MARFT to discussing its perspectives and challenges.

Powerful Capabilities for Solving Complex Agentic Tasks Agentic tasks are significantly more complex than traditional language tasks, requiring capabilities such as autonomous decision-

making, collaboration, and adaptive reasoning. While a standalone LLM-based agent may lack the breadth of skills needed to address these multifaceted challenges, MARFT empowers LaMAS to overcome these limitations. By leveraging RFT, MARFT enables the efficient decomposition of complex instructions into executable sub-tasks, distributing them among specialized agents within the LaMAS framework. This allows heterogeneous LLM-based agents to contribute their unique intelligence while dynamically sharing goals, negotiating strategies, and aligning team objectives through natural language interactions. MARFT further enhances this process by optimizing both individual agent policies and collective system performance, ensuring efficient coordination and adaptability. For example, in a logistics scenario, MARFT fine-tunes agents to collaboratively solve the task of "delivering emergency supplies" by dynamically assigning sub-tasks such as route planning, inventory allocation, and risk assessment to specialized agents, thereby maximizing the system's overall effectiveness.

Scalability and Robust Generalization MARFT offers significant scalability. Agentic multi-agent problems are highly flexible, allowing for various ways to decompose tasks and design corresponding multi-agent systems. As the agent population, environmental complexity, and task scale increase, the key challenge is to maintain efficient and stable learning performance and coordination capabilities. MARFT not only enhances the overall performance of the multi-agent system but also increases the intelligence of each agent by enabling more dynamic role-playing. Even if the system organization changes significantly, the problem can be re-decomposed to allow agents to function within a multi-agent system similar to the one they were trained in.

Enhanced Privacy and Federated Learning In a unified multi-agent system, agents do not have access to each other's local data. Instead, they contribute their unique intelligence to the system through their actions, thereby enhancing the system's collective intelligence. This setup aligns with many real-world scenarios where entities are willing to share their powerful agents but are highly concerned about data privacy. While this resembles federated learning, MARFT places greater emphasis on improving system performance through better collaboration among agents.

Integration with Blockchain Technology MARFT's decentralized nature and privacy-preserving capabilities make it a natural fit for blockchain. In blockchain systems, MARFT can enable secure and efficient multi-agent collaboration without requiring agents to share sensitive data. For example, in smart contract execution, MARFT can coordinate multiple agents to validate transactions, manage decentralized autonomous organizations (DAOs), or optimize resource allocation in decentralized finance (DeFi) platforms. The ability of MARFT to dynamically adapt to changing environments and agent populations ensures robust performance in the unpredictable and adversarial settings typical of blockchain ecosystems. Additionally, MARFT's emphasis on preserving pretrained capabilities while adapting to new tasks aligns well with the need for agents to maintain trustworthiness and reliability in blockchain-based systems. This integration opens new avenues for enhancing transparency, security, and efficiency in blockchain applications while leveraging the collective intelligence of LaMAS.

7 Open Problems

Despite the advantages and potential of MARFT paradigm, both academia and industry face numerous challenges that impede the development of more effective MARFT algorithms.

Lack of Dynamic Environments for Training A significant challenge in advancing MARFT is the absence of well-established, easy-to-implement, and scalable dynamic interactive environments designed for solving agentic tasks. There are two primary difficulties in this regard. First, creating such environments with complex agentic tasks demands substantial engineering expertise, comparable to developing fundamental environments like VirtualHome [67] or Overcooked [7] from scratch. Second, designing reward feedback mechanisms for multi-agent systems in highly dynamic environments with complex agentic tasks is exceedingly challenging. The reward signals can be

multi-dimensional, and balancing the weights among different goals is a delicate task. Although there are several "dynamic" benchmarks available, such as the Berkeley Function Calling Leaderboard [106, 65], ToolSandBox [48], and GAIA [54], converting these into dynamic environments that support MARL training remains an unresolved issue.

Low Sample Efficiency and Lack of High-Quality Synthetic Data RL, particularly on-policy RL, is known for its low sample efficiency. Algorithms like PPO and TRPO, which guarantee monotonic improvement, require frequent switching between sampling trajectories and training. This process is especially problematic and time-consuming when applied to LLMs. Potential solutions may include adopting ideas like Dyna [84] to enhance sample efficiency or developing algorithms that effectively utilize stale trajectories. Another issue stemming from low sample efficiency is the scarcity of high-quality synthetic data. Effective multi-agent trajectories need to achieve high success rates while also demonstrating efficient communication and coordination. Currently, the field lacks such data for multi-agent cold starts.

Lack of an Established MARFT Framework Developing engineering solutions that integrate LLMs and MARL is highly challenging. While there exist user-friendly, efficient, and scalable frameworks such as verl [77], OpenRLHF [32] for LLM fine-tuning and MALib [122], ReMA [96], PettingLLMs [121] for MARL training, a comprehensive framework specifically designed for MARFT is notably absent. The creation of such a framework would require significantly more effort than developing either of the aforementioned frameworks individually. This is because it must effectively integrate both LLMs and MARL components, each of which presents its own set of complexities. While this article introduces a MARFT framework, it is still in its early stages and offers significant room for further development and refinement.

Lack of a Unified Communication Mechanism or Protocol Recent advancements have led to the development of numerous agent communication protocols [110], such as MCP (Model Context Protocol) [3], A2A (Agent-to-Agent) [83], and ANP (Agent Network Protocol) [22]. These protocols are essential for enabling effective collaboration among agents and facilitating interactions with users. They also address the critical need for agents to contribute to shared goals while preserving their own data and knowledge, reflecting the future vision of a decentralized, privacy-preserving agent ecosystem. However, the proliferation of these protocols has resulted in a fragmented landscape, lacking a unified and efficient communication pipeline. This fragmentation may hinder the seamless integration and interoperability of diverse agents, particularly in dynamic and heterogeneous environments typical of LaMAS. The absence of a standardized communication framework not only complicates system design and scalability but also introduces inefficiencies and potential bottlenecks in real-world applications.

8 Conclusion

In this article, we propose a novel paradigm termed **Multi-Agent Reinforcement Fine-Tuning (MARFT)**, together with a brand-new MG called Flex-MG to better align with the LaMAS optimization in realistic scenarios, and present a universal algorithmic framework, MARFT, that integrates RFT, LaMAS, and MARL to advance the capabilities of LaMAS systems. MARFT addresses several fundamental challenges in adapting conventional MARL to LaMAS, including asynchronous agent interactions, profile-aware architecture design, heterogeneity across agents, and the dynamic organization of multi-agent systems. Through the fusion of RFT with sequential decision-making paradigms, trust-region optimization techniques, and token-level adaptation mechanisms, MARFT enables efficient coordination and policy refinement while preserving the foundational capabilities of pretrained LLMs.

Updated experiments on mathematical and coding tasks show that MARFT improves LaMAS across DeepScaler and DeepCoder settings, and often outperforms Independent PPO (MAPoRL) across out-of-domain evaluations. The role-specification ablation further suggests that MARFT can sup-

port self-evolving collaboration patterns when agents are initialized without fixed role descriptions. These findings suggest that MARFT holds significant promise for agentic tasks requiring complex, well-organized, distributed reasoning and acting.

The paradigm’s intrinsic scalability, privacy-preserving characteristics, and alignment with decentralized system architectures make it a compelling foundation for real-world deployment in domains ranging from logistics and collaborative robotics to blockchain-based ecosystems. Nevertheless, numerous theoretical and practical challenges remain. Key open problems include the development of dynamic environment designs with agentic tasks tailored for LaMAS, improvement of sample efficiency, and the absence of standardized and well-established frameworks for MARFT and a unified multi-agent communication protocol.

Future research directions may include the creation of unified benchmarks and development toolkits for MARFT, the construction of high-quality synthetic datasets to simulate multi-agent LLM interactions, and the design of hybrid learning strategies to boost data efficiency, etc. By addressing these open challenges, we envision MARFT as a stepping stone toward realizing generalizable, human-aligned, and collaboratively adaptive agent systems, paving the way for scalable AGI capable of tackling complex open-world problems with robust agentic behavior and human-like adaptability.

References

- [1] *Law of Large Numbers*, pp. 297–299. Springer New York, New York, NY, 2008. ISBN 978-0-387-32833-1. doi: 10.1007/978-0-387-32833-1_224. URL https://doi.org/10.1007/978-0-387-32833-1_224.
- [2] Apple intelligence foundation language models, 2024. URL <https://arxiv.org/abs/2407.21075>.
- [3] Anthropic. Introducing the model context protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024.
- [4] Matej Balog, Alexander L. Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. Deepcoder: Learning to write programs. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=ByldLrqlx>.
- [5] Lucian Busoniu, Robert Babuska, and Bart De Schutter. A comprehensive survey of multi-agent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(2):156–172, 2008.
- [6] CAMEL-AI.org. Owl: Optimized workforce learning for general multi-agent assistance in real-world task automation. <https://github.com/camel-ai/owl>, 2025. Accessed: 2025-03-07.
- [7] Micah Carroll, Rohin Shah, Mark K Ho, Tom Griffiths, Sanjit Seshia, Pieter Abbeel, and Anca Dragan. On the utility of learning about humans for human-ai coordination. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/f5b1b89d98b7286673128a5fb112cb9a-Paper.pdf.
- [8] Thomas Carta, Clément Romac, Thomas Wolf, Sylvain Lamprier, Olivier Sigaud, and Pierre-Yves Oudeyer. Grounding large language models in interactive environments with online reinforcement learning. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 3676–3713. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/carta23a.html>.
- [9] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent llm systems fail?, 2025. URL <https://arxiv.org/abs/2503.13657>.
- [10] Dingyang Chen, Yile Li, and Qi Zhang. Communication-efficient actor-critic methods for homogeneous markov games. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=xy_2w3J3kH.
- [11] Wei Chen, Zhiyuan Li, and Mingyuan Ma. Octopus: On-device language model for function calling of software apis, 2024. URL <https://arxiv.org/abs/2404.01549>.
- [12] K. R. Chowdhary. *Natural Language Processing*, pp. 603–649. Springer India, New Delhi, 2020. ISBN 978-81-322-3972-7. doi: 10.1007/978-81-322-3972-7_19. URL https://doi.org/10.1007/978-81-322-3972-7_19.
- [13] Haixing Dai, Yiwei Li, Zhengliang Liu, Lin Zhao, Zihao Wu, Suhan Song, Ye Shen, Dajiang Zhu, Xiang Li, Sheng Li, Xiaobai Yao, Lu Shi, Quanzheng Li, Zhuo Chen, Donglan Zhang, Gengchen Mai, and Tianming Liu. Ad-autogpt: An autonomous gpt for alzheimer’s disease infodemiology, 2023. URL <https://arxiv.org/abs/2306.10095>.

- [14] Constantinos Daskalakis, Dylan J Foster, and Noah Golowich. Independent policy gradient methods for competitive reinforcement learning. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 5527–5540. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/3b2acfe2e38102074656ed938abf4ac3-Paper.pdf.
- [15] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirog Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- [16] Thomas Degris, Patrick M. Pilarski, and Richard S. Sutton. Model-free reinforcement learning with continuous action in practice. In *2012 American Control Conference (ACC)*, pp. 2177–2182, 2012. doi: 10.1109/ACC.2012.6315022.
- [17] Will Epperson, Gagan Bansal, Victor Dibia, Adam Fourney, Jack Gerrits, Erkang Zhu, and Saleema Amershi. Interactive debugging and steering of multi-agent ai systems. 2025. doi: 10.1145/3706598.3713581.
- [18] Lutfi Eren Erdogan, Nicholas Lee, Siddharth Jha, Sehoon Kim, Ryan Tabrizi, Suhong Moon, Coleman Hooper, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Tinyagent: Function calling at the edge, 2024. URL <https://arxiv.org/abs/2409.00608>.
- [19] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution, 2023. URL <https://arxiv.org/abs/2309.16797>.
- [20] Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), Apr. 2018. doi: 10.1609/aaai.v32i1.11794. URL <https://ojs.aaai.org/index.php/AAAI/article/view/11794>.

- [21] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. Areal: A large-scale asynchronous reinforcement learning system for language reasoning, 2025. URL <https://arxiv.org/abs/2505.24298>.
- [22] Gaowei Chang. ANP - Agent Network Protocol. <https://agent-network-protocol.com/>, 2025.
- [23] Anna Goldie, Azalia Mirhoseini, Hao Zhou, Irene Cai, and Christopher D. Manning. Synthetic data generation & multi-step rl for reasoning & tool use, 2025. URL <https://arxiv.org/abs/2504.04736>.
- [24] JuraJ Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Anil Palepu, Petar Sirkovic, Artiom Myaskovsky, Felix Weissenberger, Keran Rong, Ryutaro Tanno, Khaled Saab, Dan Popovici, Jacob Blum, Fan Zhang, Katherine Chou, Avinatan Hassidim, Burak Gokturk, Amin Vahdat, Pushmeet Kohli, Yossi Matias, Andrew Carroll, Kavita Kulkarni, Nenad Tomasev, Yuan Guan, Vikram Dhillon, Eeshit Dhaval Vaishnav, Byron Lee, Tiago R D Costa, José R Penadés, Gary Peltz, Yunhan Xu, Annalisa Pawlosky, Alan Karthikesalingam, and Vivek Natarajan. Towards an ai co-scientist, 2025. URL <https://arxiv.org/abs/2502.18864>.
- [25] Evan Greensmith, Peter Bartlett, and Jonathan Baxter. Variance reduction techniques for gradient estimates in reinforcement learning. In T. Dietterich, S. Becker, and Z. Ghahramani (eds.), *Advances in Neural Information Processing Systems*, volume 14. MIT Press, 2001. URL https://proceedings.neurips.cc/paper_files/paper/2001/file/584b98aac2dddf59ee2cf19ca4ccb75e-Paper.pdf.
- [26] Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In *International conference on machine learning*, pp. 1352–1361. PMLR, 2017.
- [27] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. URL <https://arxiv.org/abs/2402.14008>.
- [28] Nicolas Heess, Dhruva TB, Srinivasan Sriram, Jay Lemmon, Josh Merel, Greg Wayne, Yuval Tassa, Tom Erez, Ziyu Wang, S. M. Ali Eslami, Martin Riedmiller, and David Silver. Emergence of locomotion behaviours in rich environments, 2017. URL <https://arxiv.org/abs/1707.02286>.
- [29] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.
- [30] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xianwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VtmBAGCN7o>.
- [31] Jian Hu. Reinforce++: A simple and efficient approach for aligning large language models, 2025. URL <https://arxiv.org/abs/2501.03262>.
- [32] Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143*, 2024.

- [33] Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. Agent-coder: Multi-agent-based code generation with iterative testing and optimisation, 2024. URL <https://arxiv.org/abs/2312.13010>.
- [34] Maximilian Hüttenrauch, Adrian Šošić, and Gerhard Neumann. Deep reinforcement learning for swarm systems. *Journal of Machine Learning Research*, 20(54):1–31, 2019. URL <http://jmlr.org/papers/v20/18-476.html>.
- [35] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- [36] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. From llms to llm-based agents for software engineering: A survey of current, challenges and future, 2024. URL <https://arxiv.org/abs/2408.02479>.
- [37] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines. 2024.
- [38] D.E. Kirk. *Optimal Control Theory: An Introduction*. Networks Series. Prentice-Hall, 1970. ISBN 9780136380986. URL <https://books.google.com.hk/books?id=cPBQAAAAMAJ>.
- [39] Prabuchandran K.J., Hemanth Kumar A.N, and Shalabh Bhatnagar. Multi-agent reinforcement learning for traffic signal control. In *17th International IEEE Conference on Intelligent Transportation Systems (ITSC)*, pp. 2529–2534, 2014. doi: 10.1109/ITSC.2014.6958095.
- [40] Vijay Konda and John Tsitsiklis. Actor-critic algorithms. In S. Solla, T. Leen, and K. Müller (eds.), *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf.
- [41] Wouter Kool, Herke van Hoof, and Max Welling. Buy 4 REINFORCE samples, get a baseline for free!, 2019. URL <https://openreview.net/forum?id=r1lgTGL5DE>.
- [42] Jakub Grudzien Kuba, Muning Wen, Linghui Meng, shangding gu, Haifeng Zhang, David Mguni, Jun Wang, and Yaodong Yang. Settling the variance of multi-agent policy gradients. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (eds.), *Advances in Neural Information Processing Systems*, volume 34, pp. 13458–13470. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/6fe6a8a6e6cb710584efc4af0c34ce50-Paper.pdf.
- [43] Jakub Grudzien Kuba, Ruiqing Chen, Muning Wen, Ying Wen, Fanglei Sun, Jun Wang, and Yaodong Yang. Trust region policy optimisation in multi-agent reinforcement learning. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=EcGGFkNTxdJ>.
- [44] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021. URL <https://arxiv.org/abs/2005.11401>.
- [45] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models, 2022. URL <https://arxiv.org/abs/2206.14858>.

- [46] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step, 2023. URL <https://arxiv.org/abs/2305.20050>.
- [47] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *Neural Information Processing Systems (NIPS)*, 2017.
- [48] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities, 2024. URL <https://arxiv.org/abs/2408.04682>.
- [49] Hao Ma, Tianyi Hu, Zhiqiang Pu, Boyin Liu, Xiaolin Ai, Yanyan Liang, and Min Chen. Coevolving with the other you: Fine-tuning llm with sequential cooperative multi-agent reinforcement learning. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 15497–15525. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/1c2b1c8f7d317719a9ce32dd7386ba35-Paper-Conference.pdf.
- [50] MAA. American invitational mathematics examination - aime. <https://maa.org/math-competitions/american-invitational-mathematics-examination-aime>, 2024. Accessed: 2024-10-04.
- [51] Zhao Mandi, Shreeya Jain, and Shuran Song. Roco: Dialectic multi-robot collaboration with large language models, 2023.
- [52] Laetitia Matignon, Guillaume J. Laurent, and Nadine Le Fort-Piat. Independent reinforcement learners in cooperative markov games: a survey regarding coordination problems. *The Knowledge Engineering Review*, 27(1):1–31, 2012. doi: 10.1017/S0269888912000057.
- [53] Laetitia Matignon, Laurent Jeanpierre, and Abdel-Ilah Mouaddib. Coordinated multi-robot exploration under communication constraints using decentralized markov decision processes. *Proceedings of the AAAI Conference on Artificial Intelligence*, 26(1):2017–2023, Sep. 2021. doi: 10.1609/aaai.v26i1.8380. URL <https://ojs.aaai.org/index.php/AAAI/article/view/8380>.
- [54] Grégoire Mialon, Clémentine Fourier, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general AI assistants. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=fibxvahvs3>.
- [55] Marvin Minsky. Steps toward artificial intelligence. *Proceedings of the IRE*, 49(1):8–30, 1961. doi: 10.1109/JRPROC.1961.287775.
- [56] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning, 2013. URL <https://arxiv.org/abs/1312.5602>.
- [57] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In Maria Florina Balcan and Kilian Q. Weinberger (eds.), *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pp. 1928–1937, New York, New York, USA, 20–22 Jun 2016. PMLR. URL <https://proceedings.mlr.press/v48/mniha16.html>.
- [58] Simon Nakibinge, Yongrui Liu, Qiang Gao, and Griffiths Anusu Luyali. Ppo improvement in different environments. In *2024 3rd International Conference on Electronics and Information Technology (EIT)*, pp. 783–788, 2024. doi: 10.1109/EIT63098.2024.10762021.

- [59] Yiwen Nie, Junhui Zhao, Feifei Gao, and F. Richard Yu. Semi-distributed resource management in uav-aided mec systems: A multi-agent federated reinforcement learning approach. *IEEE Transactions on Vehicular Technology*, 70(12):13162–13173, 2021. doi: 10.1109/TVT.2021.3118446.
- [60] Frans A. Oliehoek and Christopher Amato. *A Concise Introduction to Decentralized POMDPs*. Springer Publishing Company, Incorporated, 1st edition, 2016. ISBN 3319289276.
- [61] OpenAI. Learning to reason with LLMs, sep 2024. URL <https://openai.com/index/learning-to-reason-with-llms/>. OpenAI blog post introducing the o1 model’s reasoning capabilities.
- [62] OpenAI. Openai’s reinforcement fine-tuning research program, dec 2024. URL <https://openai.com/form/rft-research-program/>. OpenAI blog post expanding their Reinforcement Fine-Tuning Research Program.
- [63] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. URL <https://arxiv.org/abs/2203.02155>.
- [64] Chanwoo Park, Seungju Han, Xingzhi Guo, Asuman Ozdaglar, Kaiqing Zhang, and Joo-Kyung Kim. Maporl: Multi-agent post-co-training for collaborative large language models with reinforcement learning, 2025. URL <https://arxiv.org/abs/2502.18439>.
- [65] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023.
- [66] Guilherme Penedo, Anton Lozhkov, Hynek Kydliček, Loubna Ben Allal, Edward Beeching, Agustín Piqueres Lajarín, Quentin Gallouédec, Nathan Habib, Lewis Tunstall, and Leandro von Werra. Codeforces. <https://huggingface.co/datasets/open-r1/codeforces>, 2025.
- [67] Xavier Puig, Kevin Ra, Marko Boben, Jiaman Li, Tingwu Wang, Sanja Fidler, and Antonio Torralba. Virtualhome: Simulating household activities via programs. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 8494–8502, 2018.
- [68] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15174–15186, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.810. URL <https://aclanthology.org/2024.acl-long.810/>.
- [69] Qwen-Team. Qwq-32b: Unveiling the power of reinforcement learning in reasoning, March 2025. URL <https://qwenlm.github.io/blog/qwq-32b/>. Technical report introducing the QwQ-32B reasoning model with 32B parameters that rivals 671B models through multi-stage RL training.
- [70] Wei Ren, R.W. Beard, and E.M. Atkins. A survey of consensus problems in multi-agent coordination. In *Proceedings of the 2005, American Control Conference, 2005.*, pp. 1859–1864 vol. 3, 2005. doi: 10.1109/ACC.2005.1470239.
- [71] Timo Schick, Jane Dwivedi-Yu, Roberto Dessí, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: language models can teach themselves to use tools. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS ’23*, Red Hook, NY, USA, 2023. Curran Associates Inc.

- [72] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In Francis Bach and David Blei (eds.), *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pp. 1889–1897, Lille, France, 07–09 Jul 2015. PMLR. URL <https://proceedings.mlr.press/v37/schulman15.html>.
- [73] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- [74] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation, 2018. URL <https://arxiv.org/abs/1506.02438>.
- [75] Shai Shalev-Shwartz, Shaked Shammah, and Amnon Shashua. Safe, multi-agent, reinforcement learning for autonomous driving, 2016. URL <https://arxiv.org/abs/1610.03295>.
- [76] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- [77] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.
- [78] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, USA, 2008. ISBN 0521899435.
- [79] Yan Song, He Jiang, Zheng Tian, Haifeng Zhang, Yingping Zhang, Jiangcheng Zhu, Zonghong Dai, Weinan Zhang, and Jun Wang. An empirical study on google research football multi-agent scenarios. *Machine Intelligence Research*, pp. 1–22, 2024.
- [80] Peter Stone and Manuela Veloso. Multiagent systems: A survey from a machine learning perspective. *Autonomous Robots*, 8:345–383, 2000.
- [81] Vighnesh Subramaniam, Yilun Du, Joshua B. Tenenbaum, Antonio Torralba, Shuang Li, and Igor Mordatch. Multiagent finetuning: Self improvement with diverse reasoning chains, 2025. URL <https://arxiv.org/abs/2501.05707>.
- [82] Sainbayar Sukhbaatar, arthur szlam, and Rob Fergus. Learning multiagent communication with backpropagation. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016. URL https://proceedings.neurips.cc/paper_files/paper/2016/file/55b1927fdafef39c48e5b73b5d61ea60-Paper.pdf.
- [83] Rao Surapaneni, Miku Jha, Michael Vakoc, and Todd Segal. Announcing the agent2agent protocol (a2a). <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>, 2025. Accessed: 2025-04-14.
- [84] Richard S Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, 2(4):160–163, 1991.
- [85] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, 1998.
- [86] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In S. Solla, T. Leen, and K. Müller (eds.), *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.

- [87] Richard Stuart Sutton. *Temporal credit assignment in reinforcement learning*. PhD thesis, University of Massachusetts Amherst, 1984. AAI8410337.
- [88] Ming Tan. Multi-agent reinforcement learning: independent versus cooperative agents. In *Proceedings of the Tenth International Conference on International Conference on Machine Learning*, ICML’93, pp. 330–337, San Francisco, CA, USA, 1993. Morgan Kaufmann Publishers Inc. ISBN 1558603077.
- [89] Sijun Tan, Michael Luo, Justin Wong, Colin Cai, Xiaoxiang Shi, William Yuan Tang, Manan Roongta, Tianjun Zhang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deep-scaler: Effective RL scaling of reasoning models via iterative context lengthening, 2026. URL <https://openreview.net/forum?id=I6GzDCne7U>.
- [90] Weihao Tan, Wentao Zhang, Shanqi Liu, Longtao Zheng, Xinrun Wang, and Bo An. True knowledge comes from practice: Aligning large language models with embodied environments via reinforcement learning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=hILVmJ4Uvu>.
- [91] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. Magis: Llm-based multi-agent framework for github issue resolution. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 51963–51993. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/5d1f02132ef51602adf07000ca5b6138-Paper-Conference.pdf.
- [92] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033. IEEE, 2012. doi: 10.1109/IROS.2012.6386109.
- [93] Karl Tuyls and Gerhard Weiss. Multiagent learning: Basics, challenges, and prospects. *Ai Magazine*, 33(3):41–41, 2012.
- [94] Martijn van Otterlo and Marco Wiering. *Reinforcement Learning and Markov Decision Processes*, pp. 3–42. Springer Berlin Heidelberg, Berlin, Heidelberg, 2012. ISBN 978-3-642-27645-3. doi: 10.1007/978-3-642-27645-3_1. URL https://doi.org/10.1007/978-3-642-27645-3_1.
- [95] Guilherme S Varela, Alberto Sardinha, and Francisco S Melo. Networked agents in the dark: Team value learning under partial observability. *arXiv preprint arXiv:2501.08778*, 2025.
- [96] Ziyu Wan, Yunxiang Li, Xiaoyu Wen, Yan Song, Hanjing Wang, Linyi Yang, Mark Schmidt, Jun Wang, Weinan Zhang, Shuyue Hu, and Ying Wen. Rema: Learning to meta-think for llms with multi-agent reinforcement learning, 2025. URL <https://arxiv.org/abs/2503.09501>.
- [97] Jun Wang, Meng Fang, Ziyu Wan, Muning Wen, Jiachen Zhu, Anjie Liu, Ziqin Gong, Yan Song, Lei Chen, Lionel M. Ni, Linyi Yang, Ying Wen, and Weinan Zhang. Openr: An open source framework for advanced reasoning with large language models, 2024. URL <https://arxiv.org/abs/2410.09671>.
- [98] Xihuai Wang, Zheng Tian, Ziyu Wan, Ying Wen, Jun Wang, and Weinan Zhang. Order matters: Agent-by-agent policy optimization. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=Q-neeWNVv1>.
- [99] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3):279–292, 05 1992. ISSN 1573-0565. doi: 10.1007/BF00992698. URL <https://doi.org/10.1007/BF00992698>.
- [100] Gerhard Weiss. *Multiagent systems: a modern approach to distributed artificial intelligence*. MIT press, 1999.

- [101] Muning Wen, Jakub Kuba, Runji Lin, Weinan Zhang, Ying Wen, Jun Wang, and Yaodong Yang. Multi-agent reinforcement learning is a sequence modeling problem. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 16509–16521. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/69413f87e5a34897cd010ca698097d0a-Paper-Conference.pdf.
- [102] Muning Wen, Junwei Liao, Cheng Deng, Jun Wang, Weinan Zhang, and Ying Wen. Entropy-regularized token-level policy optimization for language agent reinforcement, 2024. URL <https://arxiv.org/abs/2402.06700>.
- [103] Muning Wen, Ziyu Wan, Weinan Zhang, Jun Wang, and Ying Wen. Reinforcing language agents via policy optimization with action decomposition, 2024. URL <https://arxiv.org/abs/2405.15821>.
- [104] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.*, 8(3–4):229–256, May 1992. ISSN 0885-6125. doi: 10.1007/BF00992696. URL <https://doi.org/10.1007/BF00992696>.
- [105] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL <https://arxiv.org/abs/2308.08155>.
- [106] Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Berkeley function calling leaderboard. In *Advances in Neural Information Processing Systems*, 2024.
- [107] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yaqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- [108] Yingxuan Yang, Qiuying Peng, Jun Wang, Ying Wen, and Weinan Zhang. Llm-based multi-agent systems: Techniques and business perspectives, 2024. URL <https://arxiv.org/abs/2411.14033>.
- [109] Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. Agentnet: Decentralized evolutionary coordination for llm-based multi-agent systems, 2025. URL <https://arxiv.org/abs/2504.00587>.
- [110] Yingxuan Yang, Huacan Chai, Yuanyi Song, Siyuan Qi, Muning Wen, Ning Li, Junwei Liao, Haoyi Hu, Jianghao Lin, Gaowei Chang, Weiwen Liu, Ying Wen, Yong Yu, and Weinan Zhang. A survey of ai agent protocols, 2025. URL <https://arxiv.org/abs/2504.16736>.
- [111] Yingxuan Yang, Bo Huang, Siyuan Qi, Chao Feng, Haoyi Hu, Yuxuan Zhu, Jinbo Hu, Haoran Zhao, Ziyi He, Xiao Liu, Zongyu Wang, Lin Qiu, Xuezhi Cao, Xunliang Cai, Yong Yu, and Weinan Zhang. Who’s the mvp? a game-theoretic evaluation benchmark for modular attribution in llm agents, 2025. URL <https://arxiv.org/abs/2502.00510>.
- [112] Yingxuan Yang, Chengrui Qu, Muning Wen, Laixi Shi, Ying Wen, Weinan Zhang, Adam Wierman, and Shangding Gu. Understanding agent scaling in llm-based multi-agent systems via diversity, 2026. URL <https://arxiv.org/abs/2602.03794>.

- [113] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2022. URL <https://openreview.net/forum?id=YVXaxB6L2Pl>.
- [114] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Weinan Dai, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- [115] Yu Yue, Yufeng Yuan, Qiying Yu, Xiaochen Zuo, Ruofei Zhu, Wenyuan Xu, Jiaze Chen, Chengyi Wang, TianTian Fan, Zhengyin Du, Xiangpeng Wei, Xiangyu Yu, Gaohong Liu, Juncai Liu, Lingjun Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Ru Zhang, Xin Liu, Mingxuan Wang, Yonghui Wu, and Lin Yan. Vapo: Efficient and reliable reinforcement learning for advanced reasoning tasks, 2025. URL <https://arxiv.org/abs/2504.05118>.
- [116] Zhiyuan Zeng, Qinyuan Cheng, Zhangyue Yin, Bo Wang, Shimin Li, Yunhua Zhou, Qipeng Guo, Xuanjing Huang, and Xipeng Qiu. Scaling of search and learning: A roadmap to reproduce o1 from reinforcement learning perspective, 2024. URL <https://arxiv.org/abs/2412.14135>.
- [117] Kaiqing Zhang, Zhuoran Yang, Han Liu, Tong Zhang, and Tamer Başar. Fully decentralized multi-agent reinforcement learning with networked agents, 2018. URL <https://arxiv.org/abs/1802.08757>.
- [118] Kaiyan Zhang, Runze Liu, Xuekai Zhu, Kai Tian, Sihang Zeng, Guoli Jia, Yuchen Fan, Xingtai Lv, Yuxin Zuo, Che Jiang, Ziyang Liu, Jianyu Wang, Yuru Wang, Ruotong Zhao, Ermo Hua, Yibo Wang, Shijie Wang, Junqi Gao, Xinwei Long, Youbang Sun, Zhiyuan Ma, Ganqu Cui, Lei Bai, Ning Ding, Biqing Qi, and Bowen Zhou. Marti: A framework for multi-agent llm systems reinforced training and inference, 2025. URL <https://github.com/TsinghuaC3I/MARTI>.
- [119] Weinan Zhang, Junwei Liao, Ning Li, Kounianhua Du, and Jianghao Lin. Agentic information retrieval, 2025. URL <https://arxiv.org/abs/2410.09713>.
- [120] Wanjia Zhao, Mert Yuksekgonul, Shirley Wu, and James Zou. Sirius: Self-improving multi-agent systems via bootstrapped reasoning, 2025. URL <https://arxiv.org/abs/2502.04780>.
- [121] Yujie Zhao, Lanxiang Hu, Yang Wang, Minmin Hou, Hao Zhang, Ke Ding, and Jishen Zhao. Stronger together: On-policy reinforcement learning for collaborative llms. *arXiv preprint arXiv:2510.11062*, 2025.
- [122] Ming Zhou, Ziyu Wan, Hanjing Wang, Muning Wen, Runzhe Wu, Ying Wen, Yaodong Yang, Yong Yu, Jun Wang, and Weinan Zhang. Malib: A parallel framework for population-based multi-agent reinforcement learning. *Journal of Machine Learning Research*, 24(150):1–12, 2023. URL <http://jmlr.org/papers/v24/22-0169.html>.
- [123] Yifei Zhou, Song Jiang, Yuandong Tian, Jason Weston, Sergey Levine, Sainbayar Sukhbaatar, and Xian Li. Sweet-rl: Training multi-turn llm agents on collaborative reasoning tasks, 2025. URL <https://arxiv.org/abs/2503.15478>.

A Derivation of Token-Level Sub-Optimality and the Corrections

This appendix gives the detailed derivation behind the compact token-level discrepancy equations in Section 4.1. The main idea is simple: before a whole agent action is completed, intermediate tokens receive zero environment reward. Therefore, every intra-action token transition only propagates future value through the token-level discount γ_w . If this propagation introduces extra discounting relative to the action-level Bellman target, token-level optimization is no longer exactly aligned with action-level optimization.

We first restate the token-level Bellman backups. The first branch handles an intermediate token inside the current agent response. The second branch moves from the last token of agent m to the first token of agent $m + 1$ without receiving environment reward. The third branch is reached only after the last agent finishes its last token, where the environment reward and the inter-action discount γ_a are applied.

$$\begin{aligned}
 Q^\pi(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j-1}, w_t^{m,j}) &\leftarrow \\
 &\begin{cases} \gamma_w \max_{w_t^{m,j+1}} Q^\pi(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j}, w_t^{m,j+1}), & \text{if } j < |a_t^m|, \\ \max_{w_t^{m+1,1}} Q^\pi(s_t, \mathbf{a}_t^{1:m}, w_t^{m+1,1}), & \text{if } j = |a_t^m| \text{ and } m < n, \\ R(s_t, \mathbf{a}_t) + \gamma_a \max_{w_{t+1}^{1,1}} Q^\pi(o_{t+1}, w_{t+1}^{1,1}), & \text{if } j = |a_t^m| \text{ and } m = n. \end{cases} \\
 V^\pi(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j}) &\leftarrow \begin{cases} 0 + \gamma_w V^\pi(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j+1}), & \text{if } j < |a_t^m| \\ 0 + V^\pi(o_t, \mathbf{a}_t^{1:m}, w_t^{m+1,1}), & \text{if } j = |a_t^m| \text{ and } m < n \\ R(s_t, \mathbf{a}_t) + \gamma_a V^\pi(s_{t+1}, \emptyset), & \text{if } j = |a_t^m| \text{ and } m = n \end{cases}
 \end{aligned}$$

Now fix an arbitrary time step t , agent index m , and token position j . We compare the token-level value at the partial prefix $(\mathbf{a}_t^{1:m-1}, w_t^{m,1:j})$ with the macro-level value after the whole joint action $\mathbf{a}_t^{1:n}$ has been formed. For readability, define the two accumulated intra-action discount exponents

$$\begin{aligned}
 E_1 &= \sum_{k=m}^n |a_t^k| - j - (n - m), \\
 E_2 &= \sum_{k=m}^n |a_t^k| + \sum_{k=m}^n |a_{t+1}^k| + m - j - 2n.
 \end{aligned}$$

Here, E_1 counts the token-level propagation steps from the current token to the end of the current joint action, while E_2 additionally includes the token-level expansion of the next-step value term. These are exactly the extra discount factors that do not appear in the original action-level Bellman target.

$$Q^{\pi^*}(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j-1}, w_t^j) = \gamma_w^{|a_t^m|-j} \max_{w_t^{m+1,1}} Q^{\pi^*}(s_t, \mathbf{a}_t^{1:m}, w_t^{m+1,1})$$

First, repeatedly apply the zero-reward token transitions until the remaining tokens of agent m and the later agents $m + 1, \dots, n$ are completed. All of these intermediate transitions only multiply the future target by γ_w , producing $\gamma_w^{E_1}$.

$$= \gamma_w^{E_1} \max_{w_t^{m,j:|a_t^m|}} \max_{\mathbf{a}_t^{m+1:n}} \left[R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a \max_{w_{t+1}^{1,1}} Q^{\pi^*}(s_{t+1}, w_{t+1}^{1,1}) \right]$$

Then distribute this accumulated factor across the immediate reward term and the future-state value term. The reward is obtained at the end of the current joint action, but it has already been discounted by all previous intra-action token transitions.

$$= \gamma_w^{E_1} \max_{w_t} \max_{m,j:|a_t^m|} \max_{a_t^{m+1:n}} R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a \gamma_w^{E_1} \max_{w_{t+1}^{1,1}} Q^{\pi^*}(s_{t+1}, w_{t+1}^{1,1})$$

Finally, expand the next-step token-level Q term into the next-step macro action. This contributes the remaining token-level discount, so the total exponent on the future action-level value becomes E_2 .

$$\begin{aligned} &= \gamma_w^{E_1} \max_{w_t} \max_{m,j:|a_t^m|} \max_{a_t^{m+1:n}} R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a \gamma_w^{E_2} \max_{a_{t+1}^{1:n}} Q^{\pi^*}(s_{t+1}, \mathbf{a}_{t+1}^{1:n}) \\ &= \gamma_w^{E_1} R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a \gamma_w^{E_2} \max_{a_{t+1}^{1:n}} Q^{\pi^*}(s_{t+1}, \mathbf{a}_{t+1}^{1:n}) \end{aligned}$$

We can now add and subtract the ordinary action-level Bellman target. The first underbraced term is exactly the macro-level target; the second underbraced term is the bias introduced by the token-level discounting.

$$\begin{aligned} &= \underbrace{R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a \max_{a_{t+1}^{1:n}} Q^{\pi^*}(s_{t+1}, \mathbf{a}_{t+1}^{1:n})}_{Q^{\pi^*}(s_t, \mathbf{a}_t^{1:n})} \\ &- \underbrace{\left[(1 - \gamma_w^{E_1}) R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a (1 - \gamma_w^{E_2}) \max_{a_{t+1}^{1:n}} Q^{\pi^*}(s_{t+1}, \mathbf{a}_{t+1}^{1:n}) \right]}_{\text{The Discrepancy}} \end{aligned}$$

The derivation for V follows the same pattern, but it is shorter because the state-value backup does not require the additional maximization over the next token. We again propagate through the remaining zero-reward token transitions, compare the result with the action-level value target, and isolate the discrepancy.

$$V^{\pi^*}(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j}) = \gamma_w^{|a_t^m|-j} V^{\pi^*}(s_t, \mathbf{a}_t^{1:m}, w_t^{m+1,1})$$

After all remaining tokens in the current joint action are traversed, the same accumulated exponent E_1 appears.

$$\begin{aligned} &= \gamma_w^{E_1} \left[R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a V^{\pi^*}(s_{t+1}, \emptyset) \right] \\ &= \underbrace{R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a V^{\pi^*}(s_{t+1}, \emptyset)}_{V^{\pi^*}(s_t)} \\ &- \underbrace{\left[(1 - \gamma_w^{E_1}) R(s_t, \mathbf{a}_t^{1:n}) + \gamma_a (1 - \gamma_w^{E_1}) V^{\pi^*}(s_{t+1}, \emptyset) \right]}_{\text{The Discrepancy}} \end{aligned}$$

Thus, both derivations expose the same issue: the token-level backup inserts powers of γ_w before the reward and future value are compared with the action-level Bellman target. These extra factors are not part of the original macro-level optimization objective.

The token-level value functions perfectly align with the macro-level value functions (i.e., introducing no token-level discounting bias) if and only if:

$$\begin{aligned} Q^{\pi^*}(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j-1}, w_t^{m,j}) &= Q^{\pi^*}(s_t, \mathbf{a}_t), \\ V^{\pi^*}(s_t, \mathbf{a}_t^{1:m-1}, w_t^{m,1:j}) &= V^{\pi^*}(s_t), \end{aligned}$$

meaning the discrepancy should equal 0. For general nonzero rewards and values, this requires removing the intra-action discounting bias by setting $\gamma_w = 1$. In that case, $\gamma_w^{E_1} = \gamma_w^{E_2} = 1$, both discrepancy terms vanish, and token-level MARFT preserves the same optimality condition as action-level MARFT.